第1章: 開発環境のセットアップ

> この章では、書籍管理Webアプリケーションを開発するために必要なツールをすべてインストールし、正しく動作することを確認します。プログラミングが初めての方でも迷わないよう、手順を一つひとつ丁寧に解説します。

この章で行うこと

「開発環境のセットアップ」とは、**プログラムを書いて動かすために必要なソフトウェアをパソコンにインストールする作業**です。料理に例えると、実際に料理を始める前に「包丁・まな板・鍋」を揃えるようなものです。

具体的には、以下の4つのツールをインストールします。

ツール	料理で例えると	なぜ必要か
Node.js	ガスコンロ（調理の火力源）	プログラムを動かすエンジン。これがないと何も始まりません
VS Code	作業台（まな板と包丁のセット）	プログラムを書くためのエディタ。メモ帳でも書けますが、VS Codeは間違いを指摘してくれたり、コードに色を付けてくれたりする優秀な道具です
Git	レシピノート	コードの変更履歴を記録するツール。「さっきの状態に戻したい」が簡単にできます
ターミナル	キッチン全体	すべてのツールを操作するための入口。文字を打ってパソコンに指示を出します

> **用語メモ:**
>
> - **ターミナル**（terminal：端末）／**コマンドライン**（command line：コマンド行）／**シェル**（shell：殻）／**CLI**（Command Line Interface：コマンドラインインターフェース）はほぼ同じ意味で使われます。すべて「文字を打ってパソコンに命令する画面」を指す言葉です。本書ではまとめて「ターミナル」と呼びます。
> - **GUI**（Graphical User Interface：グラフィカルユーザーインターフェース）はマウスでアイコンをクリックして操作する、いつものWindowsやMacの画面のことです。

> **所要時間の目安：** 30〜60分（ダウンロードの待ち時間を含む）

この章のゴールと地図

この章を読み終えて手を動かし終えると、次のことが**自分の手で**できるようになります。

- ターミナルを開いて `node -v` のようなコマンドを打ち、表示された結果を読み取れる
- Node.js と npm を使って、他の人が作ったパッケージ（部品）をインストールし、自分のプログラムから使える

- VS Code でフォルダを開き、コードを書いて、VS Code の中のターミナルから実行できる
- Git で「変更する → 記録する（コミット）」ができ、GitHub に送る準備ができている
- うまくいかないときに、エラーメッセージから「次にどこを確認すればよいか」を判断できる

下の図が、この章の「地図」です。上の段の4つの道具を左から順番にそろえていき、最後に動作確認をしてゴールです。下の段のターミナルは、すべての道具を操作するための「入口」なので、最初から最後までずっと使います。

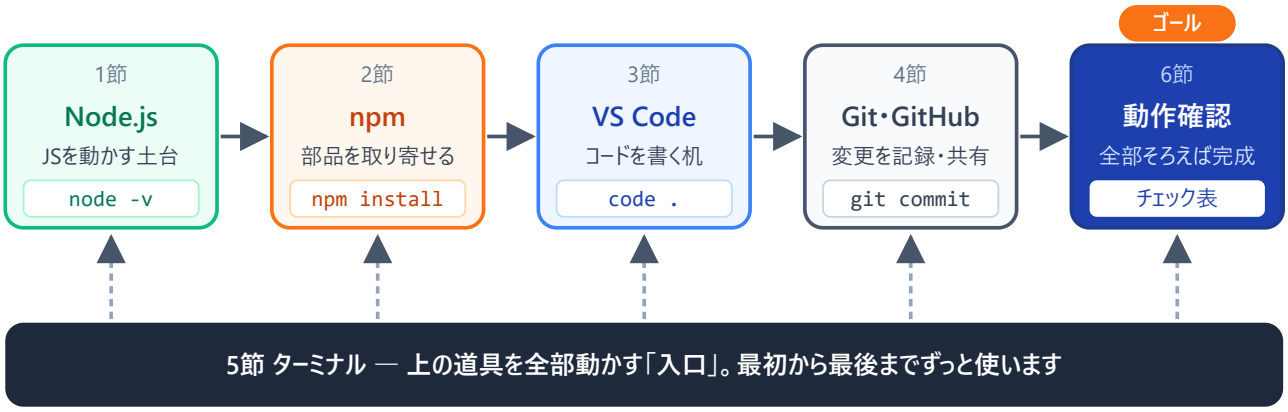


図1-1: この章の地図。ターミナルを入口にして、4つの道具を左から順にそろえていきます

> **ヒント:** ターミナル（黒い画面）を一度も使ったことがない人は、先に「5. ターミナル/コマンドラインの基礎」の 5.1 だけ読んでから 1 節に戻ると、コマンドを打つ手順で迷いにくくなります。

目次

1. Node.js のインストール
2. パッケージマネージャーの理解
3. コードエディタ (VS Code)
4. Git の基礎
5. ターミナル/コマンドラインの基礎
6. 動作確認

---

1. Node.js のインストール

1.1 Node.js とは何か

Node.js（ノードジェイエス）は、JavaScript（ジャバスクリプト：もともとはWebブラウザ上で動くプログラミング言語）をブラウザの外（サーバーやあなたのPC）で実行できるようにする**ランタイム環境**（Runtime Environment：プログラムを実行するための土台ソフトウェア）です。

もともとJavaScriptはブラウザの中だけで動く言語でしたが、2009年にNode.jsが登場したことで、サーバーサイド（server side：サーバー側、ユーザーから見えない裏側の処理）やコマンドラインツール（CLI：キーボードで文字を打って操作するプログラム）の開発にも使えるようになりました。

> **なぜNode.jsが重要なのか：** 現代のWeb開発では、プログラムを書いた後に「TypeScriptをJavaScriptに変換する」「複数のファイルを1つにまとめる」「開発用サーバーを起動する」といった様々な処理が必要です。これらはすべてNode.js上で動きます。つまり、Node.jsは**開発の土台**となるソフトウェアです。

なぜ Node.js が必要なのか

書籍管理アプリの開発では、以下の場面で Node.js を使います。

用途	説明
フロントエンド開発サーバー	React などのフレームワークをローカル（自分のPC）で動かす
パッケージ管理	npm を使ってライブラリ（他人が作った便利なコード）をインストールする
ビルドツール	TypeScript のコンパイル（変換）、バンドル（複数ファイルの結合）など
バックエンド API サーバー	Express.js などで REST API（データをやり取りする窓口）を構築する
データベース操作	Prisma などの ORM（データベースとプログラムをつなぐ翻訳ツール）を使ってデータベースとやり取りする

ブラウザの中の JavaScript と、ブラウザの外の JavaScript

「JavaScript はブラウザで動くもの」と聞いたことがあるかもしれません。では、なぜわざわざ Node.js を入れるのでしょうか。

たとえ話をします。ブラウザは「JavaScript エンジンを積んだ自動車」です。エンジンは強力ですが、乗っている人（Web サイト）が勝手に**あなたの家（PC）の中を物色できない**よう、できることが厳しく制限されています。Node.js は、**同じエンジンを車から取り外して、あなたの作業場（PC）に据え付けたもの**です。作業場では、ファイルを読み書きしたり、サーバーを起動したりと、開発に必要な力仕事が何でもできます。

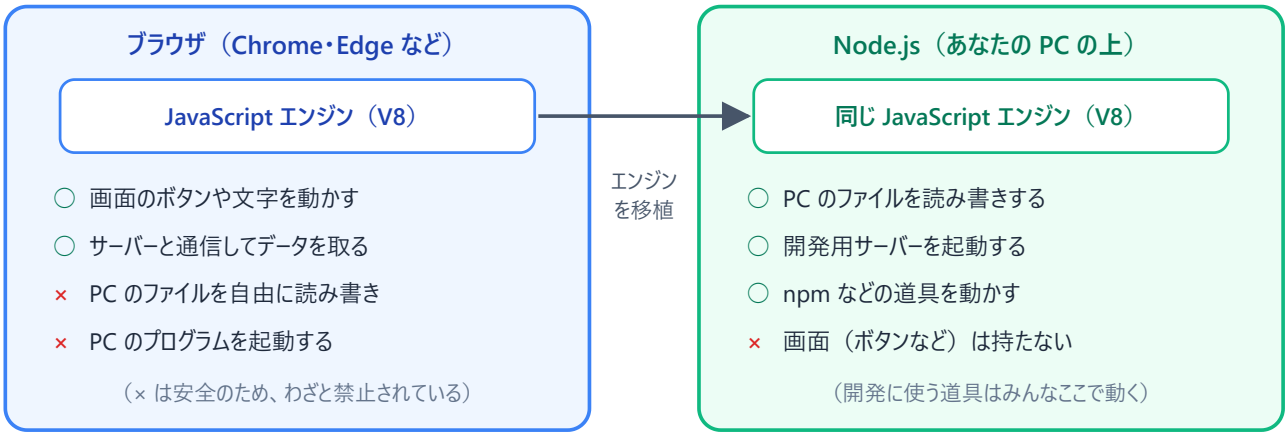


図1-2: 同じ JavaScript エンジンでも、ブラウザの中と Node.js の上では「できること」が違う

この章の後半（1.3）で、実際に Node.js から「ファイルを作って読み出す」小さなプログラムを動かして、ブラウザとの違いを体験します。

> **用語: V8**（ブイエイト）は Google が作った JavaScript エンジン（JavaScript のコードを読んで実行する心臓部）の名前です。Chrome と Node.js は、どちらもこの V8 を使っています。

開発ワークフローにおける Node.js の位置づけ



1.2 インストール方法

Node.js のインストールには2つの方法があります。

1. **公式サイトから直接インストール** — 最もシンプルな方法
2. **nvm (Node Version Manager) を使う** — 推奨される方法

> 💡 **ヒント:** 本チュートリアルでは **nvm を使ったインストールを強く推奨** します。理由は、プロジェクトによって必要な Node.js のバージョンが異なることがあり、nvm を使えばバージョンの切り替えが簡単にできるからです。

> **nvm（エヌブイエム：Node Version Manager）とは:** Node.jsの「複数バージョン管理ツール」のこと。Aプロジェクトでは Node.js 22、Bプロジェクトでは Node.js 24、というように1台のPCで異なるバージョンを使い分けられます。

nvm は「本棚の司書」のようなものです。いくつもの版（バージョン）の Node.js を棚に並べておき、「今日はこの1冊（この版）を使います」と指示すると、それを手元に出してくれます。

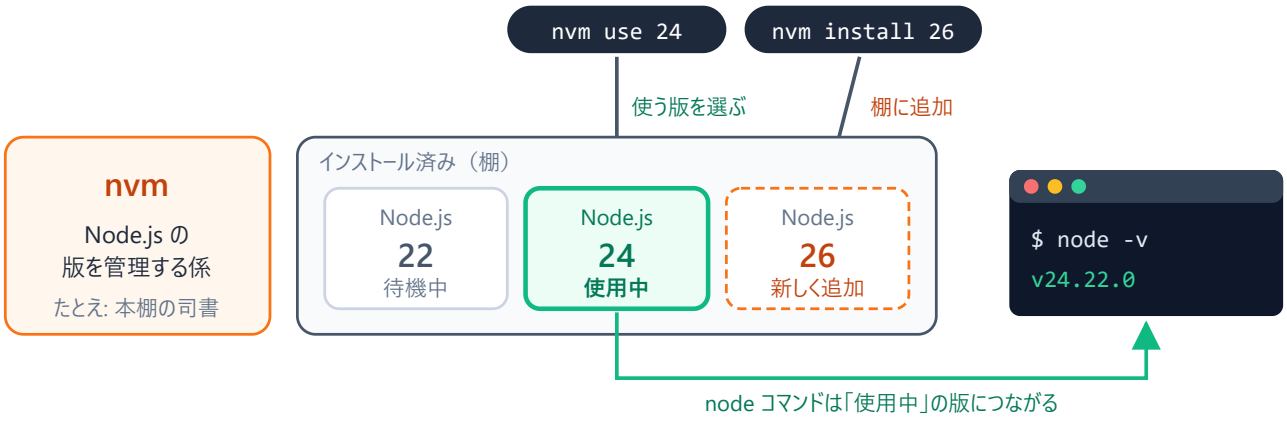


図1-3: nvm の仕組み。いくつもの版を棚に並べておき、nvm use で「いま使う1つ」を選ぶ

方法 A: nvm を使ったインストール（推奨）

Windows の場合

Windows では `nvm-windows`（正式名: NVM for Windows）を使います（Linux/Mac 用の nvm とは別のツールで、コマンドも少し違います）。2026年10月時点の最新版は **v2 系**（v2.0.1）です。v2 は v1 から全面的に作り直され、**管理者権限なしで使える**ようになりました。

手順 0: 自分の PC の種類を確認する

インストーラーは、PC の頭脳（CPU）の種類ごとに分かれています。「設定」→「システム」→「バージョン情報」を開き、「システムの種類」の欄を見てください。

「システムの種類」の表示	ダウンロードするファイル
「x64 ベース プロセッサ」	`nvm-<バージョン>-x64-setup.exe`（ほとんどの Windows PC はこちら）
「ARM ベース プロセッサ」	`nvm-<バージョン>-arm64-setup.exe`（Snapdragon 搭載 PC など）

> **注意:** 以前に公式サイト（nodejs.org）のインストーラーで Node.js を入れたことがある場合は、先に「設定」→「アプリ」→「インストールされているアプリ」から **Node.js をアンインストール**しておきましょう。2つの Node.js が同居すると、「どちらの node が動いているのか分からない」という事故が起きます（1.4 のつまずきポイントで詳しく説明します）。

手順 1: nvm-windows のダウンロード

1. nvm-windows リリースページ にアクセスします
2. 「Latest」と表示されている最新版の「Assets」欄から、手順 0 で確かめた種類の `...-setup.exe` をダウンロードします（ `.exe` はWindowsの実行ファイル形式）

手順 2: インストーラーの実行

ダウンロードしたファイルを**ダブルクリック**で起動します。**右クリック** →「**管理者として実行**」**は使いません**（理由は下のつまずきポイントで説明します）。

```
1. ライセンス（利用条件）を確認して同意し「Next」
2. インストール先などは既定（デフォルト）のまま「Next」
   （既定の場所: C:\Users\<ユーザー名>\AppData\Local\Author Software\nvm）
3. 「Install」をクリック
4. 「このアプリがデバイスに変更を加えることを許可しますか?」と聞かれたら「はい」
   （記録（ログ）の出し先を Windows に登録するための確認。「いいえ」でも nvm は使えます）
5. 「Finish」をクリック
```

> **用語メモ:**
>
> - **`C:\Users\<ユーザー名>`** : Windowsで自分の個人用フォルダ（ホームフォルダ）。`<ユーザー名>` の部分はあなたのWindowsアカウント名に置き換わります（例: `C:\Users\yuya` ）。環境変数 `%USERPROFILE%` でも同じ場所を指します。
> - **`AppData\Local`** : アプリのデータが入る隠しフォルダ。通常エクスプローラーでは見えません。環境変数 `%LOCALAPPDATA%` でも同じ場所を指します。
> - **シム**（shim：すき間に挟む「くさび」のこと）: v2 の標準の動き方。`node` と打つと、まず nvm の小さな中継プログラム（シム）が起動し、「いま使う版の Node.js」へ橋渡しします。古い v1 は**シンボリックリンク**（ファイルやフォルダの「ショートカット」のようなもの）の行き先を入れ替える方式で、その作成に管理者権限が必要でした。

> **つまずきポイント:** v2 のインストーラーを「管理者として実行」すると、nvm が**管理者アカウント用**に設定されてしまい、ふだん使っているあなたのアカウントからは正しく使えなくなります（公式ドキュメントでも「管理者としてインストールしないこと」と警告されています）。

NG（よくある間違い）:

```
インストーラーを右クリック →「管理者として実行」でインストール
→ ふだんのアカウントで開いた PowerShell で nvm が見つからない／設定が反映されない
```

OK（正しいやり方）:

```
「設定」→「アプリ」から NVM for Windows をいったんアンインストール
→ インストーラーをダブルクリックで（普通に）起動して入れ直す
```

手順 3: Node.js のインストール

インストールが終わったら、PowerShell を**新しく**開きます（スタートメニューで「PowerShell」と検索してクリック。管理者として開く必要はありません）。そして、以下を1行ずつ実行します。

> **なぜ「新しく」開くの？:** ターミナルは、起動した瞬間の設定（後で説明する「環境変数 PATH」など）を覚えたまま動き続けます。インストール前から開いていたターミナルは、新しく入った nvm の存在を知りません。

```
# nvm が使えるか確認。nvm の設定情報がずらっと表示されればインストール成功
nvm env

# インストールできる Node.js のバージョン一覧をネット経由で取得して表示する
# releases = 「公開されている版」という意味
nvm list releases

# LTS（Long Term Support：長期サポート版）の最新版をダウンロードしてインストール
# LTS版は数年間バグ修正が続く安定版。仕事ではLTSを使うのが普通
nvm install lts

# インストール済みのうち LTS 版を「現在使うバージョン」として有効化
# nvm は複数バージョンを共存できるが、同時に使うのは1つだけなので切替が必要
nvm use lts

# インストール済みの版の一覧を表示。* 印が付いているのが「いま使っている版」
nvm list

# 確認: Node.js本体のバージョンを表示。-v は --version の短縮形
node -v
# 確認: npm（次の節で説明するパッケージ管理ツール）のバージョンを表示
npm -v
```

> **インストール失敗時の典型原因:**
>
> - 「管理者として実行」でインストールした → 上のつまずきポイントの通りに入れ直す
> - インストール前から開いていたターミナルで試している → ターミナルを閉じて新しく開く
> - ウイルス対策ソフトがブロック → 一時的にオフにして再試行
> - 社内ネットワークやプロキシ → 会社支給PCの場合、IT担当者に相談
> - `node -v` は動くのに `npm -v` だけ赤いエラー（「スクリプトの実行が無効」）→ 1.4 の「PowerShell で npm だけエラーになる」を参照

> **注意（古い nvm-windows v1.x を使っている人へ）**: v1.x では `nvm install` / `nvm use` の実行に**管理者権限が必要**で、版の一覧は `nvm list available`、nvm 自体の確認は `nvm version` というコマンドでした。v2 のインストーラーを実行すると v1 の設定は自動で引き継がれる（移行される）ので、この機会に v2 へ更新するのがおすすめです。

Mac の場合

手順 1: nvm のインストール

ターミナル（macOS標準アプリの`Terminal.app`：アプリケーション → ユーティリティ にあります）を開いて以下を実行します。

```
# nvm の公式インストールスクリプトを curl でダウンロードしてシェルで実行する
# curl: ネット上のファイルを取ってくるコマンド
#   -o-  : ダウンロードしたデータをファイルに保存せず、標準出力（次のコマンド）に流す
# |     : パイプ。左コマンドの出力を右コマンドの入力につなぐ
# bash  : シェル本体。受け取ったスクリプトを解釈して実行する
# v0.40.8 の部分は nvm のバージョン（2026年10月時点の最新）
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
```

> **ヒント:** nvm の最新版の番号は、nvm の公式ページ の「Installing and Updating」に載っているコマンドで確認できます。本書より新しい番号が載っていれば、そちらを使ってください。

> **つまずきポイント:** 買ったばかりの Mac では、zsh の設定ファイル `~/.zshrc` がまだ存在しないことがあります。その状態でインストールスクリプトを実行すると、「nvm を読み込む設定」の書き込み先が見つからず、最後に次のメッセージが出て、ターミナルを開き直しても `nvm` が使えません。

NG（`~/.zshrc` が無いまま実行したときに出るメッセージ）:

```
=> Profile not found. Tried ~/.bashrc, ~/.bash_profile, ~/.zprofile, ~/.zshrc, and
~/.profile.
=> Create one of them and run this script again
```

OK（空の設定ファイルを作ってから、もう一度実行する）:

```
# 空の ~/.zshrc を作る。touch は「ファイルが無ければ空で作る」コマンド
touch ~/.zshrc

# インストールスクリプトをもう一度実行する（今度は ~/.zshrc に設定が書き込まれる）
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash
```

手順 2: シェルの再起動

インストール後、ターミナルを一度閉じて再度開きます。または以下を実行します。

> **シェル設定ファイルとは:** ターミナルを開いたときに自動で読み込まれる設定ファイル。`~/.zshrc` や `~/.bashrc` がそれ。`~` はあなたのホームフォルダを指す記号。インストーラーがこのファイルに `nvm` の読み込み行を追記しているので、再読み込みが必要。

```
# zsh（macOS Catalina以降のデフォルトシェル）を使っている場合
# source: 指定ファイルを「いま開いているシェル」で再読み込みするコマンド
# ~/.zshrc: ホームフォルダ直下にある zsh の設定ファイル
source ~/.zshrc

# bash（古いmacOSや一部Linuxのデフォルト）を使っている場合
# ~/.bashrc: ホームフォルダ直下にある bash の設定ファイル
source ~/.bashrc
```

手順 3: Node.js のインストール

```
# nvm 自体が使えるか確認。バージョン番号が表示されればインストール成功
nvm --version

# LTS（長期サポート）版のNode.jsをインストール
# --lts はロングフラグ（長い名前のオプション）。意味は Windows の `lts` と同じ
nvm install --lts

# 確認: Node.js のバージョン表示
node -v
# 確認: npm のバージョン表示
npm -v
```

> 💡 **ヒント:** Mac で Homebrew（Macの追加ソフト管理ツール）を使っている場合は `brew install nvm` でもインストールできますが、公式のインストールスクリプトを使う方が確実です。

Linux（Ubuntu/Debian）の場合

```
# システムのパッケージ情報を最新化する
# sudo  : 一時的に管理者権限で実行（Super User DO の略）
# apt   : Debian/Ubuntu系の標準パッケージ管理コマンド
# update: パッケージ一覧を最新化（実際のインストールはまだしない）
sudo apt update

# curl（ダウンロードコマンド）をインストール
# install: 指定パッケージをダウンロード＆インストール
# -y    : 確認プロンプトすべてに「yes」と自動回答するフラグ
sudo apt install curl -y

# nvm の公式インストールスクリプトをダウンロード＆実行
# curl -o- でデータを取得し、パイプ | で bash に流して実行する
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.8/install.sh | bash

# .bashrc（bash用シェル設定ファイル）を現在のシェルで再読み込み
# これで nvm コマンドが使えるようになる
source ~/.bashrc

# nvm がインストールできたか確認
nvm --version

# Node.js の LTS 版をインストール
nvm install --lts

# nvm alias: 別名（エイリアス）を設定するサブコマンド
# default node: 「ターミナルを新しく開いたとき、最新版のnodeを自動で使う」設定
# これがないと、ターミナルを開くたびに `nvm use --lts` を打つ羽目になる
nvm alias default node

# 確認
node -v
npm -v
```

方法 B: 公式サイトから直接インストール

nvm を使わない場合は、公式サイトから直接インストールできます。

1. Node.js 公式サイト にアクセスします
2. **LTS（推奨版）** をクリックしてダウンロードします
3. ダウンロードしたインストーラーを実行します

```
Windows: .msi ファイル（Microsoft Installerの略）を実行 → 画面の指示に従う
Mac:     .pkg ファイル（macOSのインストーラー形式）を実行 → 画面の指示に従う
Linux:   公式リポジトリを追加してインストール（下記参照）
```

Linux（Ubuntu/Debian）で公式リポジトリを使う場合:

```
# NodeSource（Node.js公式の配布元）リポジトリ追加スクリプトを実行する
# curl -fsSL は4つのフラグを連結:
#   -f : サーバーエラー時に静かに失敗（Fail silently）
#   -s : 進捗バーを表示しない（Silent）
#   -S : -s 中でもエラーは表示する（Show errors）
#   -L : リダイレクト（転送）に追従する（Location）
# setup_24.x : Node.js のメジャーバージョン24系（2026年10月時点の推奨LTS）を指定
# |  : 左で取得したシェルスクリプトを右のコマンドに渡す
# sudo -E bash - : 環境変数を保持(-E)したまま管理者権限のbashで実行。- は「標準入力を読む」の意味
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -

# 上のスクリプトでリポジトリ追加が済んだので、apt で nodejs をインストール
# -y : すべての確認プロンプトに自動でyes
sudo apt install -y nodejs

# 確認: バージョン表示
node -v
npm -v
```

1.3 バージョン確認

インストールが完了したら、ターミナル（Windows は PowerShell、Mac/Linux はターミナル）で以下を実行して確認します。

```
# Node.js 本体のバージョンを表示する
# -v は --version の短い書き方。多くのCLIツールで共通の慣習
node -v
# 出力例: v24.22.0
# 先頭のvは version の v。続く数字は「メジャー.マイナー.パッチ」の3階層

# npm（Node.jsに同梱されているパッケージ管理ツール）のバージョンを表示
npm -v
# 出力例: 11.19.1

# Node.js が実際にコードを実行できるか1行スクリプトでテスト
# -e "..." は eval の e。引用符内のJavaScriptコードをその場で実行するフラグ
# console.log() はカッコ内の文字を画面に出力する関数
node -e "console.log('Hello, Node.js!')"
# 出力: Hello, Node.js!
```

> 💡 **ヒント:** 本チュートリアルでは **Node.js 24（LTS）** を推奨します。最低でも **22 以上**にしてください。Next.js 自体が動く最低ラインは 20.9 ですが、Node.js 20 は 2026年4月にサポート（セキュリティ修正の提供）が終わっているため、これから使い始めるには向きません。`node -v` の表示が `v22.` か `v24.` で始まっていれば OK です。

成功すると、ターミナルには次の図のような表示が出ます。数字の細かい部分（`24.22.0` の `22.0` など）はインストールした時期によって変わりますが、**形が同じなら成功**です。

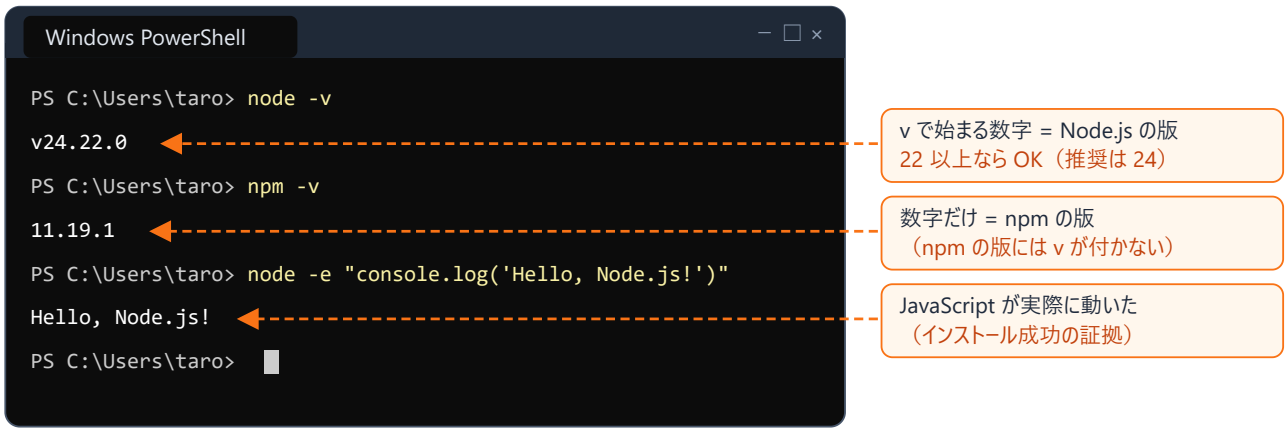


図1-4: インストールに成功したときの画面イメージ（Windows の PowerShell）。黄色が自分で打つ部分、白が表示される結果

動かしてみよう: Node.js だけができること（ファイルの読み書き）

1.1 の図1-2で「Node.js はブラウザと違って PC のファイルを読み書きできる」と説明しました。本当にそうなのか、小さなプログラムで確かめてみましょう。

> **▼ このコードがやること（先に日本語で）:** `memo.txt` というファイルを新しく作って文字を書き込み、すぐにそのファイルを読み出して画面に表示します。最後に、動いている Node.js の版と OS の種類も表示します。

準備: ターミナルで次の2行を実行し、練習用のフォルダを作ってその中に移動します（PowerShell・Git Bash・Mac のターミナルのどれでも同じです）。

```
# デスクトップに node-practice というフォルダを作る
mkdir ~/Desktop/node-practice
# 作ったフォルダの中に移動する（以降のコマンドはこのフォルダの中で動く）
cd ~/Desktop/node-practice
```

次に、このフォルダの中に `file-demo.mjs` という名前のファイルを作り、下のコードを貼り付けて保存します。VS Code（3節でインストール）がまだ無ければ、Windows のメモ帳や Mac のテキストエディットでも作れます。

```
// Node.js に最初から入っている「ファイルを読み書きする道具」を読み込む
import fs from "node:fs";

// memo.txt というファイルに文字を書き込む（無ければ新しく作られる）
fs.writeFileSync("memo.txt", "読みたい本: リーダブルコード");

// いま書いたファイルを読み出して、文字列として受け取る
const text = fs.readFileSync("memo.txt", "utf8");

// 読み出した中身を画面に表示する
console.log("memo.txt の中身 →", text);

// Node.js 自身のバージョンと、動いている OS の種類も表示する
console.log("Node.js:", process.version, "/ OS:", process.platform);
```

保存できたら、ターミナルで次のコマンドを実行します。

```
# node の後ろにファイル名を書くと、そのファイルの JavaScript を実行する
node file-demo.mjs
```

▼ **実行結果**（Windows で実行した場合）

```
memo.txt の中身 → 読みたい本: リーダブルコード
Node.js: v24.22.0 / OS: win32
```

実行後にフォルダの中を見ると、`memo.txt` というファイルが本当に増えています。Mac では `OS:` の後ろが `darwin`、Linux では `linux` と表示されます。

▼ コードを1つずつ分解して解説

解説1: 道具を読み込む `import`

```
import fs from "node:fs";
```

- `import 名前 from "場所"` は「よそにある道具を、この名前で使えるように持ってくる」という書き方です。
- `"node:fs"` は Node.js に最初から入っている（インストール不要の）ファイル操作の道具です。`fs` は file system（ファイルシステム）の略です。
- ブラウザには `node:fs` が存在しないので、このコードはブラウザでは動きません。これが図1-2の「×」の正体です。

> **用語: `.mjs`**（エムジェイエス）:「このファイルは `import` を使う新しい書き方（ES Modules：イーエスモジュール）の JavaScript です」と Node.js に伝える拡張子です。拡張子が `.js` だと、置き場所の設定によっては `import` がエラーになることがあるため、この章の単体サンプルでは `.mjs` を使います（Next.js のプロジェクトでは設定済みなので `.ts` / `.tsx` で書けます）。

解説2: ファイルに書き込む `writeFileSync`

```
fs.writeFileSync("memo.txt", "読みたい本: リーダブルコード");
```

- 1つ目の値（引数）が**ファイル名**、2つ目が**書き込む中身**です。
- ファイルが無ければ新しく作り、すでにあれば**中身をまるごと上書き**します。
- 名前の最後の `Sync`（シンク：synchronous＝同期の略）は「書き終わるまで次の行に進まない」という意味です。

解説3: ファイルを読み出す `readFileSync`

```
const text = fs.readFileSync("memo.txt", "utf8");
```

- `"utf8"` は文字コード（文字を数字に置き換えるルール）の指定です。これを付けると、読み出した中身を**普通の文字列**として受け取れます。
- 付け忘れると、人間には読めない数字の並び（バイト列）として受け取ってしまいます。

解説4: Node.js が用意している情報 `process`

```
console.log("Node.js:", process.version, "/ OS:", process.platform);
```

- `process` は、Node.js が最初から用意している「いま動いているプログラムの情報のかたまり」です。
- `process.version` は Node.js の版（`node -v` と同じ文字列）、`process.platform` は OS の種類（Windows なら `win32`）です。

- `console.log()` にカンマ区切りで複数の値を渡すと、間に空白を入れて1行に並べて表示します。

> **つまずきポイント:** メモ帳で保存したら、ファイル名が `file-demo.mjs.txt` になっていた——これは本当によくある失敗です。Windows は標準で拡張子（ファイル名の最後の `.txt` など）を隠すので、見た目では気づけません。その状態で実行すると、Node.js は「そんなファイルは無い」と言います。

NG（ファイル名が `file-demo.mjs.txt` になっている状態で実行）：

```
node:internal/modules/cjs/loader:1568
  throw err;
  ^

Error: Cannot find module 'C:\Users\<ユーザー名>\Desktop\node-practice\file-demo.mjs'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1564:15)
     （「at ...」で始まる行が続くが省略）
    at node:internal/main/run_main_module:33:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v24.22.0
```

OK（本当のファイル名を確かめて、正しい名前にする）：

```
# フォルダの中身を一覧表示して、本当のファイル名を確かめる
ls
# file-demo.mjs.txt と表示されたら、名前を file-demo.mjs に変更する
mv file-demo.mjs.txt file-demo.mjs
# もう一度実行する
node file-demo.mjs
```

> **ヒント:** エクスプローラーの「表示」→「表示」→「ファイル名拡張子」にチェックを入れると、拡張子が常に見えるようになります。開発をするなら必ずオンにしておきましょう。メモ帳で保存するときは「ファイルの種類」を「すべてのファイル (*.*)」にしてから名前を付けると、`.txt` が勝手に付きません。

1.4 トラブルシューティング

「node」コマンドが見つからない

```
'node' は、内部コマンドまたは外部コマンド、
操作可能なプログラムまたはバッチ ファイルとして認識されていません。
```

上はコマンドプロンプトの表示です。同じ失敗でも、ターミナルによって表示が違います。どれも意味は「`node` という名前のプログラムが見つからない」です。

PowerShell の場合（日本語版 Windows での実際の表示）：

```
node : 用語 'node' は、コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの名前として認識されません。名前が正しく記述されていることを確認し、パスが含まれている場合はそのパスが正しいことを確認してから、再試行してください。
発生場所 行:1 文字:1
+ node -v
+ ~~~~
    + CategoryInfo          : ObjectNotFound: (node:String) [],
CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

Git Bash の場合:

```
bash: node: command not found
```

Mac（zsh）の場合:

```
zsh: command not found: node
```

原因: Node.js への実行ファイルの場所（パス）が **環境変数 PATH** に登録されていません。

> **環境変数 PATH とは:** OS（WindowsやMac）が「コマンドが入力されたとき、どのフォルダから実行ファイルを探すか」を記したリスト。`node` と打ったときに `C:\Program Files\nodejs\node.exe` が見つけられないと、上記エラーになります。

> **用語: 環境変数**（かんきょうへんすう）: OS が全部のプログラムに配っている「共通のメモ書き」です。`PATH` はその中の1枚で、「プログラムを探しに行くフォルダの一覧」が書かれています。

PATH の動きを図にすると、次のようになります。ターミナルは `node` と打たれると、PATH に書かれたフォルダを**上から順番に**のぞいて `node`（Windows では `node.exe`）というファイルを探し、**最初に見つけたもの**を実行します。最後まで見つからなければ「command not found」です。

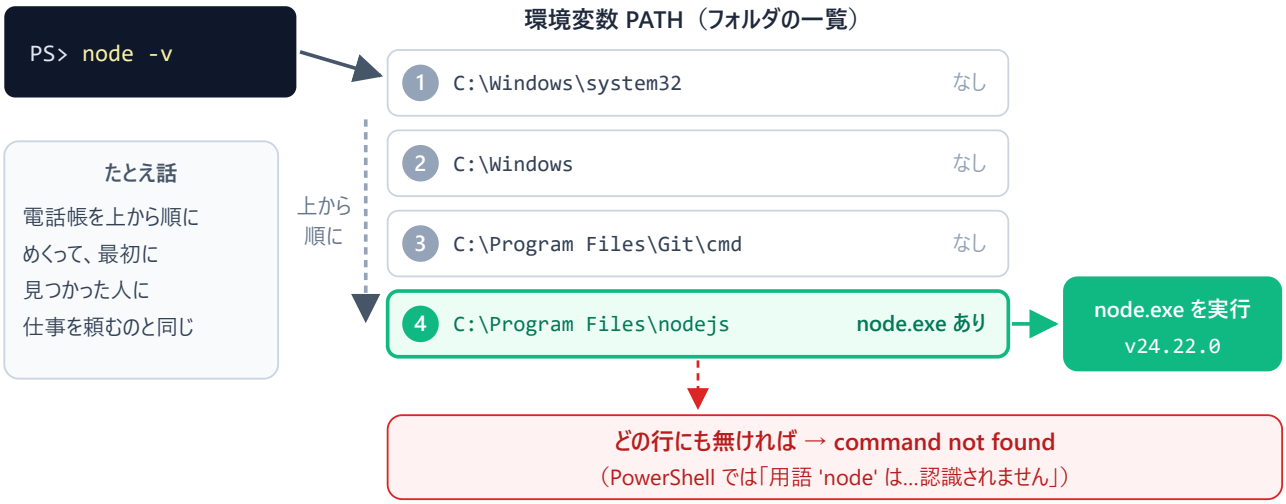


図1-5: PATH の仕組み。PATH に並んだフォルダを上から順にのぞき、最初に見つかったプログラムを実行する

実験: PATH の探し方を自分の手で再現する

「上から順に探す」を、Node.js の小さなプログラムで再現してみましょう。ターミナルが毎回こっそりやっていることが、目に見えるようになります。

> **▼ このコードがやること（先に日本語で）**：環境変数 PATH をフォルダごとに切り分け、上から1つずつ「この中に node（または指定した名前）のプログラムはあるか？」を調べて結果を表示します。最初に見つかった時点で探すのをやめます。

1.3 で作った `node-practice` フォルダに、`find-command.mjs` という名前で保存してください。

```
// ファイルの有無を調べる道具と、パス（住所）を組み立てる道具を読み込む
import fs from "node:fs";
import path from "node:path";

// 探すコマンド名。node find-command.mjs git のように指定できる（省略時は node）
const name = process.argv[2] ?? "node";

// 環境変数 PATH を区切り文字で分割して「フォルダの一覧」にする
// path.delimiter は Windows なら ";"、Mac/Linux なら ":"
const folders = process.env.PATH.split(path.delimiter);

// Windows では実行ファイルに .exe や .cmd が付くので、その候補も用意する
const extensions = process.platform === "win32" ? [".exe", ".cmd"] : [""];

console.log(`「${name}」を PATH の ${folders.length} 個のフォルダから順番に探します`);

// フォルダを上から1つずつ調べ、最初に見つかった時点で探すのをやめる
for (const [index, folder] of folders.entries()) {
  const hit = extensions
    .map((ext) => path.join(folder, name + ext))
    .find((file) => fs.existsSync(file));

  if (hit) {
    console.log(`  ${index + 1}. ${folder} … 見つかった！ → ${hit}`);
    process.exit(0);
  }
  console.log(`  ${index + 1}. ${folder} … なし`);
}

// 最後まで見つからなかった場合（これが command not found の正体）
console.log(`どこにもありませんでした → '${name}' は command not found になります`);
```

```
# node を探す（名前を省略すると node を探す）
node find-command.mjs
# git を探す
node find-command.mjs git
# わざと打ち間違えた名前を探す
node find-command.mjs nodee
```

▼ **実行結果**（説明のために、PATH を図1-5と同じ4つのフォルダだけにした Windows で実行したもの）

```
「node」を PATH の 4 個のフォルダから順番に探します
  1. C:\Windows\system32 … なし
  2. C:\Windows … なし
  3. C:\Program Files\Git\cmd … なし
  4. C:\Program Files\nodejs … 見つかった！ → C:\Program Files\nodejs\node.exe
「git」を PATH の 4 個のフォルダから順番に探します
  1. C:\Windows\system32 … なし
  2. C:\Windows … なし
  3. C:\Program Files\Git\cmd … 見つかった！ → C:\Program Files\Git\cmd\git.exe
「nodee」を PATH の 4 個のフォルダから順番に探します
  1. C:\Windows\system32 … なし
  2. C:\Windows … なし
  3. C:\Program Files\Git\cmd … なし
  4. C:\Program Files\nodejs … なし
どこにもありませんでした → 'nodee' は command not found になります
```

あなたの PC で実行すると、PATH にはもっとたくさん（20〜40個ほど）のフォルダが並ぶはずです。nvm で入れた場合は、`node` が見つかる場所も nvm のフォルダになります。

▼ コードを1つずつ分解して解説

解説1: コマンドの後ろに書いた値を受け取る `process.argv`

```
const name = process.argv[2] ?? "node";
```

- `process.argv` は、ターミナルで打った内容を空白で区切って並べたもの（配列）です。`node find-command.mjs git` なら、0番目が node 本体の場所、1番目がこのファイルの場所、**2番目が `git`** です。
- `??`（ヌル合体演算子）は「左が無ければ右を使う」という意味です。何も指定しなければ `"node"` を探します。

解説2: PATH を「フォルダの一覧」に切り分ける

```
const folders = process.env.PATH.split(path.delimiter);
```

- `process.env.PATH` で環境変数 PATH の中身（1本の長い文字列）を取り出せます。
- PATH はフォルダ同士が区切り文字でつながっているので、`split()` で切り分けて一覧（配列）にします。区切り文字は Windows が `;`（セミコロン）、Mac/Linux が `:`（コロン）で、`path.delimiter` が OS に合わせて自動で選んでくれます。

解説3: 上から順に調べ、見つかったら終わる

```
for (const [index, folder] of folders.entries()) {
  const hit = extensions
    .map((ext) => path.join(folder, name + ext))
    .find((file) => fs.existsSync(file));
```

- `for ... of` で、一覧の**先頭から順番に**フォルダを1つずつ取り出します。`entries()` を使うと、何番目か（`index`）も一緒に受け取れます。
- `path.join()` はフォルダ名とファイル名を正しい区切り文字（Windows は `\`）でつなぎます。
- `fs.existsSync()` は「そのファイルが存在するか」を `true` / `false` で答えます。
- 見つかったら `process.exit(0)` でプログラムを終了します。**最初に見つかったものだけが使われる**——これが本物のターミナルと同じ動きです。

> **ヒント:** この「最初に見つかったものが勝つ」というルールのせいで、Node.js を2つ入れてしまうと「古い方が先に見つかって動く」という事故が起きます（このあとの「Node.js が2つ入っている」で扱います）。

> **つまずきポイント:** インストールが終わったのに、**インストール前から開いていたターミナル**で `node -v` を試して「見つからない」と悩む人がとても多いです。ターミナルは起動した瞬間の PATH を覚えたまま動くので、インストールで PATH が書き換わっても、開きっぱなしのターミナルには伝わりません。VS Code の中のターミナルも同じで、**VS Code ごと**再起動が必要です。

NG（インストール前から開いていたターミナルで、そのまま確認する）:

```
# Node.js のインストーラーを実行 → 完了。でも、このターミナルは古い PATH のまま
node -v
# → node : 用語 'node' は、コマンドレット、関数、…として認識されません。
```

OK（ターミナルをいったん全部閉じ、新しく開いてから確認する）:

```
# 新しく開いたターミナルは、書き換わった後の PATH を読み込んで起動する
node -v
# → v24.22.0
```

解決方法（Windows）:

```
# 1. 現在の PATH をセミコロン区切りで分割して見やすく表示
# $env:PATH : PowerShellで環境変数PATHを参照する書き方（$env: は環境変数のプレフィックス）
# -split ";" : 文字列をセミコロンで分割して配列にする演算子
$env:PATH -split ";"

# 2. Node.js のインストール先を確認
#    通常は以下のいずれか
#    - C:\Program Files\nodejs\
#    - C:\Users\<ユーザー名>\AppData\Roaming\nvm\<バージョン>\

# 3. 環境変数を設定（システムのプロパティ → 環境変数 → PATH に追加）
#    または PowerShell で一時的に追加（再起動で消える）：
# += は「右辺を足して代入」する演算子。既存の PATH に新パスを末尾追加
$env:PATH += ";C:\Program Files\nodejs\"

# 4. ターミナルを再起動して再度確認
node -v
```

解決方法（Mac/Linux）：

```
# 1. node コマンドの実体がどこにあるか調べる
# which: 指定したコマンドの実行ファイルのフルパスを表示
which node
# または
# ls: フォルダ内のファイル一覧表示。ここではファイル単体を指定し、存在チェックに使う
ls /usr/local/bin/node

# 2. nvm を使っている場合、シェル設定ファイルに以下が追加されているか確認
# cat: ファイル内容を画面に表示
# | : パイプ。左の出力を右の入力に渡す
# grep "NVM": 「NVM」という文字列を含む行だけ抽出するコマンド
cat ~/.bashrc | grep NVM
# または zsh の場合
cat ~/.zshrc | grep NVM

# 以下のような行があるはず:
# export NVM_DIR="$HOME/.nvm"
# [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

# 3. なければ手動で追加
# echo "...": 文字列を出力するコマンド
# >> : 追記リダイレクト。出力をファイル末尾に追加（> は上書き、>> は追記）
# シングルクォート '...' で囲むと、$変数 を展開せず文字通り書き込める
echo 'export NVM_DIR="$HOME/.nvm"' >> ~/.zshrc
echo '[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"' >> ~/.zshrc
# 設定ファイルを再読み込みして、その場で反映
source ~/.zshrc
```

Node.js のバージョンが古い

```
# 現在のバージョン確認
node -v
# 出力例: v18.20.8
# ↑ 古すぎる（本書の推奨は 22 以上。18 はすでにサポート終了）

# nvm を使っている場合: LTS版を取得 → 切り替え
# 最新LTSをダウンロード＆インストール
nvm install --lts
# 「いま使うNode.js」をLTSに切り替え
nvm use --lts

# nvm を使っていない場合
# → 公式サイトから最新 LTS 版をダウンロードして再インストール
```

PowerShell で npm だけエラーになる（スクリプトの実行が無効）

`node -v` は動くのに、`npm -v` だけが赤い文字のエラーになることがあります。Windows の PowerShell でとてもよく起きる現象です。

▼ 実際の表示（日本語版 Windows の PowerShell）：

```
npm : このシステムではスクリプトの実行が無効になっているため、ファイル C:\Program
Files\nodejs\npm.ps1 を読み込むことができません。詳細については、「about_Execution_Policies」
(https://go.microsoft.com/fwlink/?LinkID=135170) を参照してください。
発生場所 行:1 文字:1
+ npm -v
+ ~~~
    + CategoryInfo          : セキュリティ エラー: (: ) []、PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

原因: Node.js には `npm.ps1` という PowerShell 用の小さなスクリプト（命令を書き並べたファイル）が入っていて、PowerShell で `npm` と打つとこれが動きます。ところが Windows は、安全のために**「スクリプトの実行をすべて禁止する」設定（実行ポリシー: Restricted）**になっていることがあり、そのせいで止められています。`node` はスクリプトではなく普通のプログラム（`node.exe`）なので、影響を受けません。

> **用語: 実行ポリシー**（Execution Policy）: PowerShell が「どんなスクリプトなら実行してよいか」を決める安全設定です。

解決方法: 自分のアカウントだけ、「この PC で作られたスクリプトは実行してよい（ネットから来たものは署名つきだけ）」という設定（RemoteSigned）に変えます。PowerShell で次を実行します（管理者として開く必要はありません）。

```
# 今の設定を確認する。CurrentUser の行が Undefined や Restricted なら変更が必要
Get-ExecutionPolicy -List

# 自分のアカウント（CurrentUser）だけ、実行ポリシーを RemoteSigned に変更する
#  「実行ポリシーを変更しますか?」と聞かれたら Y を入力して Enter
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

設定を変えたら PowerShell を開き直し、もう一度 `npm -v` を実行します。

▼ 実行結果

```
11.19.1
```

> **ヒント:** 設定を変えたくない場合は、`npm` の代わりに `npm.cmd` と打っても動きます（`npm.cmd -v` で `11.19.1` のように表示されます）。また、Git Bash やコマンドプロンプトではこのエラーは起きません。

Node.js が2つ入っている（どちらが動いているか分からない）

公式サイトのインストーラーで入れたあとに nvm でも入れた——という場合に起きます。「`nvm use 24` したのに `node -v` が古い版を表示する」といった不思議な現象の原因になります。図1-5のとおり、ターミナルは**PATH で先に見つかった方**を動かすからです。

確認方法: 「`node` という名前のプログラムが、PATH のどこに何個あるか」を全部表示します。

```
# Windows（PowerShell）: where.exe は PATH から探し、見つかった場所を全部表示する
#  （PowerShell では where だけだと別のコマンドになるので、必ず .exe まで打つ）
where.exe node
```

```
# Mac / Linux / Git Bash: -a（all）を付けると、見つかった場所を全部表示する
which -a node
```

NG（2か所で見つかる。Mac での表示例）:

```
/usr/local/bin/node
/Users/taro/.nvm/versions/node/v24.22.0/bin/node
```

OK（nvm のものだけが見つかる。Mac での表示例）:

```
/Users/taro/.nvm/versions/node/v24.22.0/bin/node
```

解決方法: どちらか一方（本書では nvm）にそろえます。Windows なら「設定」→「アプリ」→「インストールされているアプリ」から、**公式インストーラーで入れた Node.js**（`where.exe node` で `C:\Program Files\nodejs\node.exe` と表示される方）をアンインストールし、ターミナルを開き直してから、もう一度 `where.exe node` で確かめます。nvm-windows v2 で入れた版も同じ一覧に表示されることがあるので、消す前に名前とバージョンをよく確認してください。

nvm コマンドが見つからない（Mac/Linux）

```
# シェル設定ファイルに NVM の設定があるか確認
# zsh の場合
# -A2: 「After 2」マッチ行の後ろ2行も一緒に表示するフラグ。設定行は3行構成なので必要
cat ~/.zshrc | grep -A2 NVM

# 設定がなければ追加
# cat >> file << 'EOF' ... EOF はヒアドキュメント構文。
# EOF までの複数行をまとめてファイル末尾に追記する。
# 'EOF'（シングルクォート付き）にすると、$変数 を展開せず文字通り書き込める
cat >> ~/.zshrc << 'EOF'
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
EOF

# 設定を反映: 現在のシェルで .zshrc を再読み込み
source ~/.zshrc
```

npm install 時に EACCES エラー（Mac/Linux）

```
npm ERR! Error: EACCES: permission denied
```

> **EACCESとは:** Error: ACCESS の略で「アクセス権限がない」というUNIX系OSの定型エラー。書き込み権限のないフォルダに書き込もうとしたときに出ます。

```
# nvm を使っていればこのエラーは通常発生しません
# グローバルインストール先を、書き込み権限がある場所に変更する手順:

# 1. ホームフォルダに専用フォルダを作る
# mkdir: フォルダ作成。~ はホームフォルダ
mkdir ~/.npm-global

# 2. npm の設定でグローバルインストール先（prefix）を上で作ったフォルダに変更
# npm config set <キー> <値> : 設定を書き換えるコマンド
npm config set prefix '~/.npm-global'

# 3. PATH に上記フォルダの bin（実行ファイル置き場）を追加するよう .bashrc に追記
# $PATH: 既存のPATHを保持しつつ前方に新しいパスを足す書き方
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc

# 4. .bashrc を再読み込みして反映
source ~/.bashrc
```

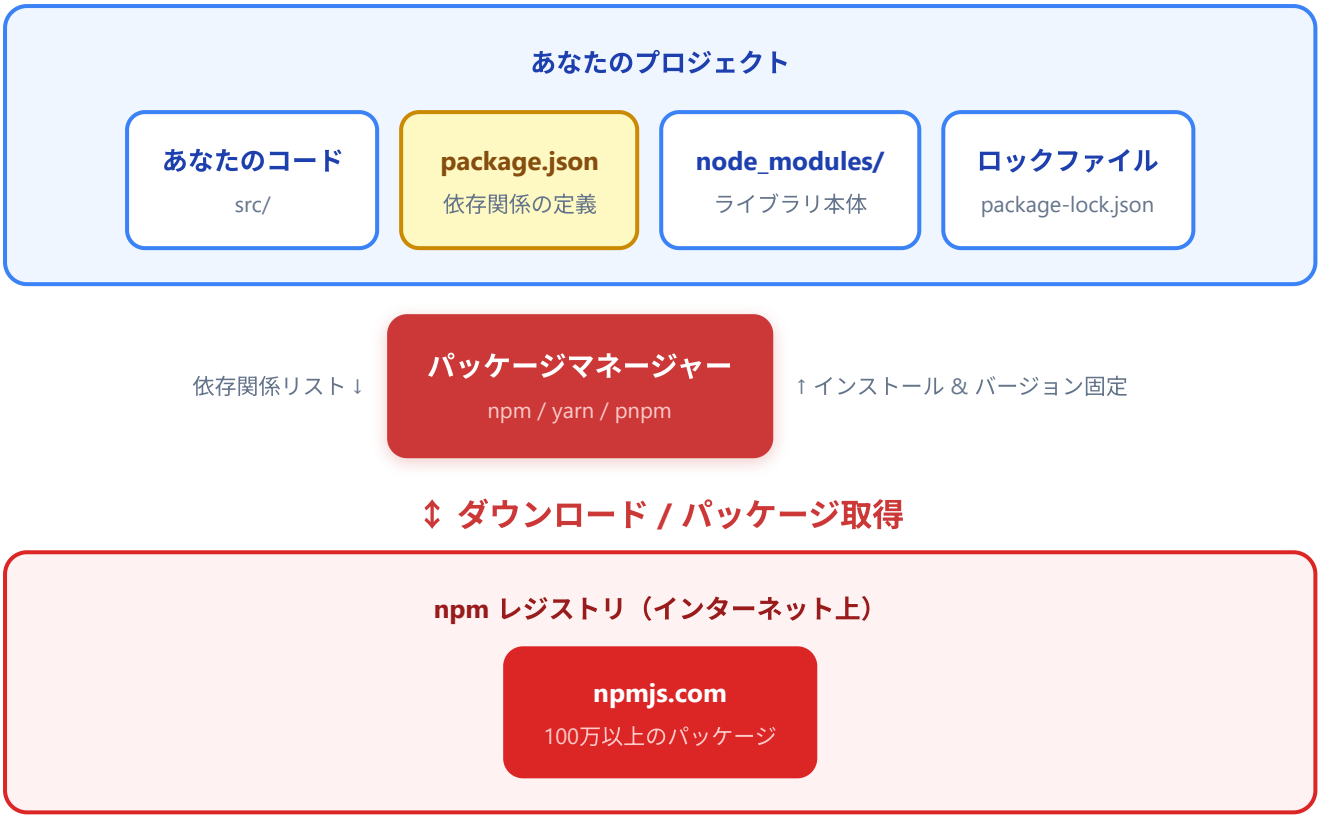
---

2. パッケージマネージャーの理解

2.1 パッケージマネージャーとは

パッケージマネージャーは、プロジェクトで使うライブラリ（パッケージ：他人が作って公開した便利なコード部品）のインストール、更新、削除を管理するツールです。たとえば、「React」や「Express」などのライブラリをコマンド一つでインストールできます。

> **用語メモ:**
>
> - **パッケージ**（package）／**ライブラリ**（library）：他人が作って公開してくれた、再利用可能なコード部品の集合。
> - **依存関係**（dependency：ディペンデンシー）：「Aを動かすためにBが必要」というつながりのこと。Aは「Bに依存している」と言う。



> **レジストリ**（registry：登録簿）とは: パッケージが世界中から集まる「公式サイト＝中央倉庫」。npm レジストリ（npmjs.com）には100万以上のパッケージが登録されており、`npm install` するとここからダウンロードされます。

2.2 npm vs yarn vs pnpm 比較

特徴	npm	yarn	pnpm
開発元	npm, Inc.（GitHub/Microsoft）	Meta（旧Facebook）	コミュニティ
Node.js 同梱	はい（デフォルト）	いいえ（別途インストール）	いいえ（別途インストール）
インストール速度	普通	速い	最も速い
ディスク使用量	多い	多い	少ない（ハードリンク）
ロックファイル	`package-lock.json`	`yarn.lock`	`pnpm-lock.yaml`
ワークスペース対応	v7 以降対応	対応	対応
学習コスト	低い	低い	やや高い
初心者おすすめ度	★★★★★	★★★★☆	★★★☆☆

> 💡 **ヒント**: 本チュートリアルでは **npm** を使用します。Node.js をインストールすると自動的に npm もインストールされるため、追加のセットアップが不要です。

2.3 package.json の理解

`package.json` はプロジェクトの「設計図」のようなファイルです。プロジェクトの名前、バージョン、使用するライブラリなどの情報がすべて記載されています。JSON（JavaScript Object Notation：構造化データの記述形式）というフォーマットで書きます。

```jsonc
{
  // プロジェクト名。英小文字とハイフン推奨
  "name": "book-management-app",
  // 自分のプロジェクトのバージョン（後述のセマンティックバージョニング形式）
  "version": "1.0.0",
  // プロジェクトの簡単な説明文（人間向け）
  "description": "書籍管理 Web アプリケーション",
  // このパッケージを他から読み込む時のエントリー（入口）ファイル
  "main": "index.js",
  // `npm run <キー>` で実行できるショートカット集
  "scripts": {
    //   npm run dev → vite（開発サーバー起動）コマンドを実行
    "dev": "vite",
    //   npm run build → 本番用にビルド（最適化変換）
    "build": "vite build",
    //   npm run start → node で server.js を起動
    "start": "node server.js",
    //   npm run test → vitest でテスト実行
    "test": "vitest"
  },
  // 本番でも必要なライブラリ。アプリ実行時に呼ばれるもの
  "dependencies": {
    //   React本体
    "react": "^18.2.0",
    //   ReactをDOM（ブラウザ）に描画するパッケージ
    "react-dom": "^18.2.0",
    //   Webサーバーフレームワーク
    "express": "^4.18.2"
  },
  // 開発時だけ必要。本番デプロイには含めなくてOK
  "devDependencies": {
    //   ビルドツール
    "vite": "^5.0.0",
    //   テストツール
    "vitest": "^1.0.0",
    //   TypeScriptコンパイラ
    "typescript": "^5.3.0"
  }
}
```

> **注意:** 上の例は、説明のために `//` のコメントを書き込んでいます。**本物の package.json にはコメントを書けません**（JSON という形式がコメントを認めていないため）。自分のファイルにコピーするときは、コメントの行を消してください。詳しくはこの節の最後の「つまずきポイント」で、実際のエラーと一緒に説明します。

各フィールドの意味:

フィールド	説明
`name`	プロジェクト名（小文字、ハイフン区切り）
`version`	プロジェクトのバージョン（セマンティックバージョニング）
`description`	プロジェクトの説明文
`scripts`	`npm run <名前>` で実行できるコマンドの定義
`dependencies`	本番環境でも必要なライブラリ
`devDependencies`	開発時のみ必要なライブラリ（テストツール、ビルドツール等）

> **セマンティックバージョニング**（semantic versioning：意味のあるバージョン番号付け、略してSemVer）：`メジャー.マイナー.パッチ` の3桁形式。たとえば `18.2.3` は メジャー18・マイナー2・パッチ3。
>
> - **メジャー**: 互換性が壊れる大きな変更があったとき増える
> - **マイナー**: 後方互換のある機能追加で増える
> - **パッチ**: バグ修正のみで増える

> **つまずきポイント:** package.json は「JavaScript っぽい見た目」ですが、中身は**JSON**という、もっと厳しいルールの形式です。`//` のコメントや、最後の項目の後ろの余計なカンマ（`,`）があるだけで、npm はファイルを読めずに止まります。

NG（コメントを書いた package.json で `npm install` を実行）：

```
{
  // プロジェクト名
  "name": "json-ng",
  "version": "1.0.0"
}
```

```
npm error code EJSONPARSE
npm error JSON.parse Invalid package.json: JSONParseError: Expected property name or
'}' in JSON at position 4 (line 2 column 3) while parsing near "{\n  // プロジェクト名\n
\"name\":..."
npm error JSON.parse Failed to parse JSON data.
npm error JSON.parse Note: package.json must be actual JSON, not just JavaScript.
npm error A complete log of this run can be found in: C:\Users\<ユーザー名
>\AppData\Local\npm-cache\_logs\2026-10-15T11_32_31_630Z-debug-0.log
```

NG（最後の項目の後ろにカンマがある package.json で `npm install` を実行）：

```
{
  "name": "json-ng",
  "version": "1.0.0",
}
```

```
npm error code EJSONPARSE
npm error JSON.parse Invalid package.json: JSONParseError: Expected double-quoted
property name in JSON at position 45 (line 4 column 1) while parsing near
"...\"version\": \"1.0.0\",\n}\n"
npm error JSON.parse Failed to parse JSON data.
npm error JSON.parse Note: package.json must be actual JSON, not just JavaScript.
npm error A complete log of this run can be found in: C:\Users\<ユーザー名
>\AppData\Local\npm-cache\_logs\2026-10-15T11_32_33_267Z-debug-0.log
```

OK（コメントも余計なカンマも無い package.json）：

```
{
  "name": "json-ng",
  "version": "1.0.0"
}
```

```
up to date, audited 1 package in 486ms

found 0 vulnerabilities
```

エラー文の `(line 2 column 3)` は「2行目の3文字目がおかしい」という意味です。**エラーには場所が書いてある**ので、まずはそこを見に行く癖をつけましょう。最後の行の `Note: package.json must be actual JSON, not just JavaScript.`（package.json は JavaScript ではなく正式な JSON でなければならない）も、そのままヒントになっています。

バージョン指定の記号

記号	意味	例
`^`	メジャーバージョン固定（マイナー・パッチは最新）	`^18.2.0` → 18.x.x の最新
`~`	メジャー・マイナー固定（パッチのみ最新）	`~18.2.0` → 18.2.x の最新
なし	完全固定	`18.2.0` → 18.2.0 のみ
`*`	任意のバージョン（非推奨）	`*` → 何でもOK

表だけだと分かりにくいので、数直線にしてみます。横軸がバージョンの古い→新しい順で、帯が「その書き方なら npm がインストールしてよい範囲」です。

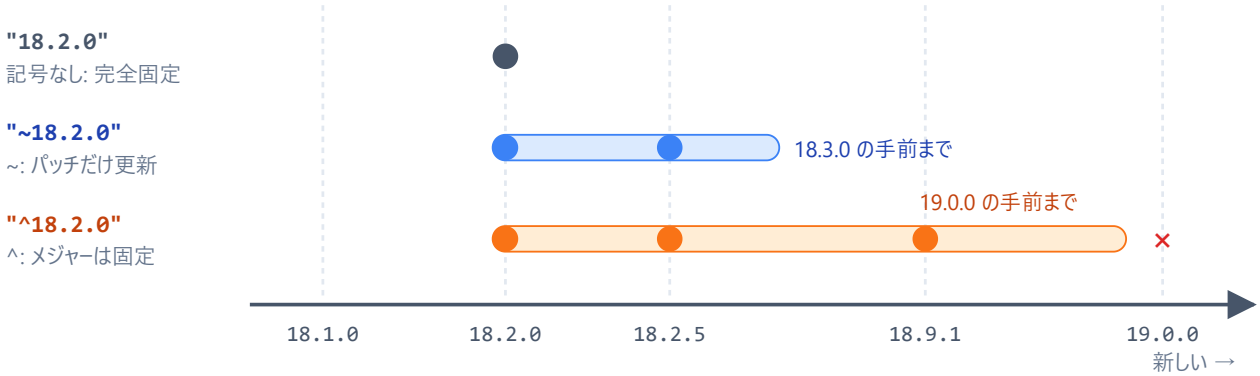


図1-7: バージョン指定の記号と「インストールしてよい範囲」。どの書き方でも、指定より古い 18.1.0 は対象外

`^`（キャレット）が npm の標準です。`npm install` でパッケージを入れると、package.json には自動で `^` 付きの番号が書かれます。「メジャー番号が同じ＝互換性が保たれているはず」なので、その範囲の新しい版なら受け入れる、という考え方です。

確かめてみよう: 範囲に入るかを `semver` で判定する

> **▼ このコードがやること（先に日本語で）**: npm と同じルールでバージョンを比べる道具（ `semver` ）を一時的に呼び出し、「並べた版のうち、指定した範囲に入るもの」だけを表示させます。図1-7の答え合わせです。

```
# ^18.2.0 の範囲に入る版だけを表示する
npx --yes semver -r "^18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0

# ~18.2.0 の範囲に入る版だけを表示する
npx --yes semver -r "~18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0

# 記号なし（完全固定）の場合
npx --yes semver -r "18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0
```

▼ 実行結果（3つのコマンドを順に実行したもの）

```
$ npx --yes semver -r "^18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0
18.2.0
18.2.5
18.9.1
$ npx --yes semver -r "~18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0
18.2.0
18.2.5
$ npx --yes semver -r "18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0
18.2.0
```

図1-7の丸印と、表示された版がぴったり一致していますね。

▼ コードを1つずつ分解して解説

解説1: インストールせずに一時的に使う `npx`

```
npx --yes semver
```

- `npx`（エヌピーエックス）は npm と一緒に入っているコマンドで、「パッケージを**インストールせずに**、一時的にダウンロードして実行する」ためのものです。一度きりしか使わない道具に向いています。
- 初めて使うパッケージだと「ダウンロードしてよいですか？」と聞かれます。`--yes` はその質問に自動で「はい」と答えるフラグです。
- 第6章で Next.js のプロジェクトを作るときにも、`npx create-next-app@latest` という形で同じ `npx` を使います。

解説2: 範囲を指定する `-r` と、比べたい版の並び

```
-r "^18.2.0" 18.1.0 18.2.0 18.2.5 18.9.1 19.0.0
```

- `-r` は range（範囲）の `r` です。続く `"^18.2.0"` が範囲です。`^` や `~` はシェルが特別扱いすることがあるので、`"` で囲んでおくと安全です。
- その後ろに空白区切りで並べた版のうち、範囲に入るものだけが1行ずつ表示されます。1つも入らないときは何も表示されません。

2.4 node_modules とは

`node_modules` フォルダは、`npm install` を実行したときにダウンロードされるライブラリの実体が格納される場所です。

```
my-project/
├── node_modules/          ← ライブラリの実体（数千ファイル）
│   ├── react/             ← React本体のフォルダ
│   ├── react-dom/         ← ReactのDOM連携部分
│   ├── express/           ← Express本体
│   └── ... 数百のパッケージ（依存ライブラリの依存ライブラリも含まれる）
├── package.json           ← 依存関係の定義（自分が書くファイル）
├── package-lock.json      ← バージョンの完全固定（npmが自動生成）
└── src/                   ← あなたのコードを置く場所
```

> ⚠️ **注意:** `node_modules` フォルダは非常に大きくなります（数百MB になることもあります）。**Git にはコミットしないでください。**後の章で設定する `.gitignore` ファイルで除外します。

> 💡 **ヒント:** `node_modules` を削除してしまっても、`npm install` を実行すれば `package.json` と `package-lock.json` の情報をもとにすべて復元できます。

3つのファイル・フォルダの関係を、料理の「買い出し」にたとえて整理します。`package.json` は**買い物メモ**（「牛乳を1本。銘柄はだいたいこれ」）、`package-lock.json` は**レシート**（「○○牛乳 1000ml を 1本買った」と正確に記録したもの）、`node_modules` は**冷蔵庫の中身**（実物）です。



図1-6: package.json・package-lock.json・node_modules の関係。Git に入れるのは「メモ」と「レシート」だけ

なぜ `package-lock.json` まで必要なのでしょうか。メモ（`^2.1.3`）だけだと「2系のどれか」としか決まらないので、来月あなたの友人が `npm install` したときには `2.1.9` が入るかもしれません。レシートがあれば、**全員がまったく同じ版**をそろえられます。「自分の PC では動くのに、友人の PC では動かない」という事故を防ぐ仕組みです。

> **つまずきポイント:** `node_modules` を Git の記録に入れてしまう失敗です。中身は数千〜数万個のファイルになることがあり、GitHub へのアップロードがとても遅くなったり、容量オーバーで失敗したりします。`.gitignore`（ジットイグノア：Git に「このファイルは無視して」と伝えるファイル）に書いておけば、Git は `node_modules` を見なくなります。

次の例は、このあと 2.5 の「やってみよう」で作る練習用フォルダで、`git init`（4節で説明する「Git の記録を始める」コマンド）を実行した直後の状態です。

NG（`.gitignore` が無い状態の `git status --short`）：`node_modules/` まで記録の候補に出てしまう

```
?? index.mjs
?? node_modules/
?? package-lock.json
?? package.json
```

OK（`.gitignore` に1行書いてから `git status --short`）：

```
# .gitignore というファイルを作り、「node_modules/ を無視して」と1行書く
echo "node_modules/" > .gitignore
# 状態をもう一度確認する。--short は1ファイル1行の短い表示
git status --short
```

```
?? .gitignore
?? index.mjs
?? package-lock.json
?? package.json
```

`node_modules/` が一覧から消えました。行頭の `??` は「Git がまだ記録していない新しいファイル」という意味です（4節で詳しく扱います）。なお、第6章で使う `create-next-app` は、`node_modules/` を除外する `.gitignore` を最初から作ってくれます。

2.5 基本的な npm コマンド

```
# プロジェクトの初期化（カレントフォルダに package.json を新規作成）
# -y は --yes の短縮形。質問にすべてデフォルト値で「はい」と答えるフラグ
# これがないと「name は？」「version は？」と対話的に聞かれる
npm init -y

# 指定パッケージを dependencies としてインストール
# install を i と短く書いてもOK（npm i react）
# 自動で package.json と package-lock.json に追記される
npm install react

# 開発時のみ必要なパッケージを devDependencies としてインストール
# --save-dev は -D とも書ける。「dev に保存」の意味
# テストツールやビルドツールはこちらに入れるのが慣習
npm install --save-dev vitest

# 引数なし: package.json に書かれた全パッケージをまとめてインストール
# 別の人のプロジェクトを git clone した直後に最初にやる操作
npm install

# 指定パッケージを削除（package.json からも消える）
# uninstall は remove や rm でも同じ動作
npm uninstall react

# scripts セクションに定義したコマンドを実行
# 例: package.json の "dev": "vite" を実行する
npm run dev

# 直接インストール済みのトップレベルパッケージ一覧を表示
# --depth=0 は「依存の深さ0階層まで」つまり自分が直接入れたものだけ表示
# これを付けないと、依存の依存も全部出てきて画面が埋まる
npm list --depth=0

# 古くなったパッケージを一覧表示。Current / Wanted / Latest 列で確認可能
# outdated = 「時代遅れの」「古い」という意味
npm outdated
```

やってみよう: パッケージを1つ入れて、自分のコードから使う

コマンドを眺めるだけでは身に付かないので、実際に手を動かします。ここでは `ms`（エムエス）という、とても小さなパッケージを使います。「2週間」のような**人間の言葉の時間**と、プログラムがよく使う**ミリ秒**（**1000分の1秒**）を相互に変換してくれる部品です。書籍管理アプリでいえば「貸出期限まであと何日？」のような計算で役立ちます。

> ▼ **このコードがやること（先に日本語で）**：練習用フォルダに package.json を作り、npm で `ms` パッケージをダウンロードします。それから、`ms` を使って「14日をミリ秒に」「ミリ秒を分や時間に」変換するプログラムを動かします。

手順1: 練習用のフォルダを作り、package.json を作る

```
# 練習用フォルダを作って、その中に移動する
mkdir ~/Desktop/npm-practice
cd ~/Desktop/npm-practice

# package.json を新しく作る（-y で質問をすべて既定値にする）
npm init -y
```

▼ 実行結果（`npm init -y`）

```
Wrote to C:\Users\<ユーザー名>\Desktop\npm-practice\package.json:

{
  "name": "npm-practice",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

`name` にはフォルダ名がそのまま入りました。

手順2: パッケージをインストールする

```
# npm レジストリから ms をダウンロードして node_modules に入れる
npm install ms
```

▼ 実行結果

```
added 1 package, and audited 2 packages in 6s

found 0 vulnerabilities
```

`added 1 package` は「パッケージを1つ追加した」、`found 0 vulnerabilities` は「既知の脆弱性（ぜいじゃくせい：セキュリティ上の弱点）は見つからなかった」という意味です。ここで package.json を開くと、末尾に次の部分が**自動**で追加されています。

```
  "dependencies": {
    "ms": "^2.1.3"
  }
```

手順3: パッケージを使うプログラムを書いて動かす

同じフォルダに `index.mjs` という名前で次のファイルを作ります。

```
// node_modules にインストールした ms パッケージを読み込む
//  ("ms" とだけ書けば、Node.js が node_modules の中から探してくれる)
import ms from "ms";

// 返却期限「2週間」をミリ秒（1000分の1秒）に変換する
console.log("2週間 =", ms("14 days"), "ミリ秒");

// 逆に、ミリ秒の数字を人間が読みやすい長さに変換する
console.log("600000ミリ秒 =", ms(600000));
console.log("3600000ミリ秒 =", ms(3600000, { long: true }));
```

```
# index.mjs を実行する
node index.mjs
```

▼ 実行結果

```
2週間 = 1209600000 ミリ秒
600000ミリ秒 = 10m
3600000ミリ秒 = 1 hour
```

▼ コードを1つずつ分解して解説

解説1: パッケージ名だけで読み込める理由

```
import ms from "ms";
```

- 1.3 の `"node:fs"` は Node.js に最初から入っている道具でしたが、`"ms"` は**npm でダウンロードしてきた**道具です。
- `"./"` や `"../"` で始まらない名前を書くと、Node.js は**同じフォルダの** `node_modules` **の中**からその名前のフォルダを探します。だから `npm install` を忘れると「見つからない」エラーになります。

解説2: 文字列を渡すとミリ秒の数字が返る

```
console.log("2週間 =", ms("14 days"), "ミリ秒");
```

- `ms("14 days")` は「14日」をミリ秒に変換した数字 `1209600000` を返します（14日 × 24時間 × 60分 × 60秒 × 1000）。
- プログラムの世界では、時間の長さをミリ秒の数字で扱うことがとても多いので、こうした変換部品が重宝されます。

解説3: 数字を渡すと、読みやすい文字列が返る

```
console.log("3600000ミリ秒 =", ms(3600000, { long: true }));
```

- 数字を渡すと逆向きに変換し、`"10m"`（10分）のような短い文字列を返します。
- 2つ目に `{ long: true }`（オプション：動きを調整する追加の指定）を渡すと、`"1 hour"` のように省略しない書き方になります。
- このように「同じ関数でも、渡す物によって動きが変わる」使い方は、パッケージの**説明書**（**README**）に書いてあります。npm の公式サイト（npmjs.com）でパッケージ名を検索すると読めます。

scripts に自分専用のコマンドを登録する

package.json の `scripts` には、よく使うコマンドに**短いあだ名**を付けて登録できます。第6章以降では `npm run dev`（開発サーバーの起動）を何度も使いますが、その正体がこれです。

> **▼ このコードがやること（先に日本語で）**：package.json の `scripts` に `"start": "node index.mjs"` を書き足し、`npm run start` と打つだけで `node index.mjs` が実行されるようにします。

package.json の `scripts` の部分を、次のように書き換えて保存します（`"test"` の行の最後に `,` を足すのを忘れずに）。

```
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node index.mjs"
  },
```

```
# scripts の start に登録したコマンドを実行する
npm run start
```

▼ 実行結果

```
> npm-practice@1.0.0 start
> node index.mjs

2週間 = 1209600000 ミリ秒
600000ミリ秒 = 10m
3600000ミリ秒 = 1 hour
```

- 最初の2行（`>` で始まる行）は npm が「これから `npm-practice` の `start` を実行します。中身は `node index.mjs` です」と教えてくれている部分です。
- `start` と `test` だけは特別扱いで、`run` を省略して `npm start` と打っても同じ結果になります。それ以外（`dev` など）は `npm run dev` のように `run` が必要です。

> **ヒント:** package.json を手で書き換える代わりに、`npm pkg set scripts.start="node index.mjs"` というコマンドでも同じ1行を追加できます。手で書き換えるとカンマの付け忘れ（2.3 のつまずきポイント）が起きやすいので、慣れてきたら便利です。

> **つまずきポイント:** `npm run dev` や `npm install` は、**package.json があるフォルダの中**で実行しないと動きません。ターミナルを開き直したときなどに、うっかりプロジェクトの外（ホームフォルダなど）で実行してしまうのは、初心者が最もよくやる失敗の1つです。

NG（package.json が無いフォルダで `npm run dev` を実行）：

```
npm error code ENOENT
npm error syscall open
npm error path C:\Users\<ユーザー名>\Desktop\empty\package.json
npm error errno -4058
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory,
open 'C:\Users\<ユーザー名>\Desktop\empty\package.json'
npm error enoent This is related to npm not being able to find a file.
npm error enoent
npm error A complete log of this run can be found in: C:\Users\<ユーザー名
>\AppData\Local\npm-cache\_logs\2026-10-15T11_32_21_006Z-debug-0.log
```

OK（今いる場所を確かめて、プロジェクトのフォルダに移動してから実行）：

```
# 今いる場所を確認する（ここが empty など、プロジェクトの外になっていないか）
pwd
# package.json がある、プロジェクトのフォルダに移動する
cd ~/Desktop/npm-practice
# package.json があることを目で確かめる
ls
# あらためて実行する
npm run start
```

`ENOENT` は "Error NO ENTry"（そんな項目は無い）の略で、「ファイルやフォルダが見つからない」という意味の定番エラーです。`Could not read package.json`（package.json を読めなかった）の一文を見たら、**まず `pwd` で現在地を確認**しましょう。

2.6 パッケージ管理のフロー

開発者 | npm | package.json | package-lock.json | node_modules/ | npm レジストリ

1. **npm init -y** — 開発者 → npm
2. **package.json 作成** — npm → package.json
3. **npm install react** — 開発者 → npm
4. **react の最新バージョンを問い合わせ** — npm → npm レジストリ
5. **react@18.2.0 + 依存パッケージ情報** — npm レジストリ → npm（応答）
6. **react 本体をダウンロード・展開** — npm → node_modules/
7. **dependencies に "react": "^18.2.0" を追記** — npm → package.json
8. **正確なバージョンとハッシュを記録** — npm → package-lock.json

別の開発者がプロジェクトに参加した場合

9. **npm install** — 開発者 → npm
10. **ロックファイルを読み込み** — npm → package-lock.json
11. **記録されたバージョンを取得** — npm → npm レジストリ
12. **同じバージョンをインストール** — npm → node_modules/

ロックファイルにより全員が同じバージョンを使える

---

3. コードエディタ (VS Code)

3.1 VS Code とは

Visual Studio Code（VS Code）は、Microsoft が開発した **無料** のコードエディタ（プログラムを書くための専用テキストエディタ）です。軽量でありながら強力な機能を持ち、豊富な拡張機能によってあらゆるプログラミング言語に対応できます。Web 開発において最も人気のあるエディタの一つです。

3.2 インストール手順

Windows

1. VS Code 公式サイト にアクセス
2. 「Download for Windows」をクリック
3. ダウンロードした `.exe` ファイルを実行
4. インストールオプション: - 「**PATHへの追加」にチェック** を入れる（重要） - 「エクスプローラーのファイルコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利：ファイルを右クリックして直接VS Codeで開ける） - 「エクスプローラーのディレクトリコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利：フォルダを右クリックして開ける）
5. 「インストール」をクリック

> ⚠️ **注意:** 「PATHへの追加」にチェックを入れ忘れると、ターミナルから `code` コマンドで VS Code を起動できません。忘れた場合は再インストールしてください。

Mac

1. VS Code 公式サイト にアクセス
2. 「Download for Mac」をクリック
3. ダウンロードした `.zip` ファイルを展開（ダブルクリックで自動展開）
4. `Visual Studio Code.app` を「アプリケーション」フォルダにドラッグ
5. `code` コマンドを使えるようにする: - VS Code を起動 - `Cmd + Shift + P` でコマンドパレット（VS Code内のコマンド検索窓）を開く - 「Shell Command: Install 'code' command in PATH」を選択

Linux（Ubuntu/Debian）

```
# Microsoft の GPG キー（電子署名の鍵）とリポジトリ（配布元）を追加する手順

# 1. Microsoftの公開鍵をネット経由で取得 → バイナリ形式に変換 → ファイル保存
# wget -qO- : URLからダウンロードして標準出力に流す
#   -q : quiet（進捗バーを表示しない）
#   -O- : 出力先を標準出力に指定（ハイフンが標準出力の意味）
# gpg --dearmor : テキスト形式の鍵をバイナリ形式に変換
# > packages.microsoft.gpg : 変換結果をファイルに保存（上書きリダイレクト）
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor >
packages.microsoft.gpg

# 2. 鍵ファイルを所定の場所へインストール（権限付き）
# install : ファイルを所定権限でコピーする専用コマンド
#   -D : 必要なら親ディレクトリも作成
#   -o root : 所有者を root に
#   -g root : グループを root に
#   -m 644 : 権限を 644（所有者=読み書き、他=読みのみ）に
sudo install -D -o root -g root -m 644 packages.microsoft.gpg
/etc/apt/keyrings/packages.microsoft.gpg

# 3. リポジトリ情報を apt のソースリストに追加
# echo "..." : 文字列を出力
# | sudo tee <ファイル> : 管理者権限で出力をファイルに書き込む
#    （sudo は echo に効かないのでこの書き方が必要）
# > /dev/null : tee の標準出力を捨てる（画面に二重表示しないため）
echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg]
https://packages.microsoft.com/repos/code stable main" | sudo tee
/etc/apt/sources.list.d/vscode.list > /dev/null

# 4. 一時ファイルを削除
# rm : ファイル削除
# -f : 存在しなくてもエラーにしない（force）
rm -f packages.microsoft.gpg

# 5. パッケージ一覧を更新してインストール
sudo apt update
sudo apt install code -y

# 6. インストール確認
# --version : バージョン情報を表示するロングフラグ
code --version
```

3.3 推奨拡張機能

VS Code のサイドバーから拡張機能アイコン（四角が4つのアイコン）をクリックし、以下の拡張機能を検索してインストールしてください。

必須の拡張機能

拡張機能名	ID	用途	重要度
Japanese Language Pack	`MS-CEINTL.vscode-language-pack-ja`	VS Code の日本語化	必須
ESLint	`dbaeumer.vscode-eslint`	JavaScript/TypeScript のコード品質チェック	必須
Prettier - Code formatter	`esbenp.prettier-vscode`	コードの自動フォーマット	必須
ES7+ React/Redux/React-Native snippets	`dsznajder.es7-react-js-snippets`	React のコードスニペット	必須

強く推奨する拡張機能

拡張機能名	ID	用途	重要度
TypeScript Importer	`pmneo.tsimporter`	import 文の自動補完	推奨
Auto Rename Tag	`formulahendry.auto-rename-tag`	HTML/JSX タグの自動リネーム	推奨
Path Intellisense	`christian-kohler.path-intellisense`	ファイルパスの自動補完	推奨
GitLens	`eamodio.gitlens`	Git の履歴・変更を可視化	推奨
Error Lens	`usernamehw.errorlens`	エラーをエディタ上にインライン表示	推奨
Thunder Client	`rangav.vscode-thunder-client`	API テスト（Postman の代替）	推奨

あると便利な拡張機能

拡張機能名	ID	用途	重要度
Material Icon Theme	`PKief.material-icon-theme`	ファイルアイコンの見た目を改善	任意
indent-rainbow	`oderwat.indent-rainbow`	インデントを色分け表示	任意
Bracket Pair Color DLW	`BracketPairColorDLW.bracket-pair-color-dlw`	対応する括弧を色分け	任意
Code Spell Checker	`streetsidesoftware.code-spell-checker`	スペルチェック	任意

> 💡 **ヒント**: 拡張機能は、コマンドラインから一括インストールすることもできます。GUIで一つずつ検索する手間が省けます。

```
# code --install-extension <ID> : 指定IDの拡張機能をインストール
# 日本語化
code --install-extension MS-CEINTL.vscode-language-pack-ja
# コード品質チェック
code --install-extension dbaeumer.vscode-eslint
# 自動フォーマッター
code --install-extension esbenp.prettier-vscode
# React用スニペット
code --install-extension dsznajder.es7-react-js-snippets
# Git可視化
code --install-extension eamodio.gitlens
# エラーをインライン表示
code --install-extension usernamehw.errorlens
```

```
# インストール済みの拡張機能のIDを一覧表示して、入ったかどうかを確かめる
code --list-extensions
```

3.4 基本的な使い方

VS Code の画面の見方

VS Code を開くと、画面は大きく5つの場所に分かれています。下の図は、2.5 で作った `npm-practice` フォルダを開き、`index.mjs` を編集しながら、下のターミナルで実行したところです。



1 アクティビティバー　2 サイドバー　3 エディター　4 パネル（ターミナル）　5 ステータスバー

図1-8: VS Code の画面イメージ（日本語化済み。2.5 の練習用フォルダを開いたところ）

No.	場所	何をする所か
1	アクティビティバー	左端の縦長のアイコン列。上から「エクスプローラー（ファイル一覧）」「検索」「ソース管理（Git）」「拡張機能」などを切り替えます
2	サイドバー	1 で選んだものの中身を表示します。上の図はエクスプローラーで、開いたフォルダの中のファイルが並んでいます
3	エディター	コードを書く場所です。上のタブで、開いているファイルを切り替えます
4	パネル（ターミナル）	VS Code の中にあるターミナルです。`Ctrl + ` ` で表示/非表示を切り替えます。**開いたフォルダの場所から始まる**のが便利な点です
5	ステータスバー	いまの Git のブランチ名（`main`）、エラーや警告の数、カーソルの位置、文字コードなどが表示されます

キーボードショートカット（Windows / Mac）

操作	Windows	Mac
コマンドパレット	`Ctrl + Shift + P`	`Cmd + Shift + P`
ファイル検索	`Ctrl + P`	`Cmd + P`
全文検索	`Ctrl + Shift + F`	`Cmd + Shift + F`
ターミナルの表示/非表示	`Ctrl + ` `	`Cmd + ` `
サイドバーの表示/非表示	`Ctrl + B`	`Cmd + B`
行の複製	`Shift + Alt + ↓`	`Shift + Option + ↓`
行の移動	`Alt + ↑/↓`	`Option + ↑/↓`
複数カーソル	`Ctrl + Alt + ↑/↓`	`Cmd + Option + ↑/↓`
名前の一括変更	`F2`	`F2`
定義へ移動	`F12`	`F12`
保存	`Ctrl + S`	`Cmd + S`
全保存	`Ctrl + K, S`	`Cmd + Option + S`

フォルダを開いて作業を始める

```
# 指定パスのフォルダをVS Codeで開く
# code は VS Code を起動するCLIコマンド（インストール時に PATH を通した）
# /path/to/my-project は開きたいフォルダの絶対パス
code /path/to/my-project

# カレントフォルダ（いま自分がいるフォルダ）をVS Codeで開く
# . は「現在のディレクトリ」を表す特殊な記号
code .
```

> **つまずきポイント:** エクスプローラーでファイル（ `index.mjs` など）をダブルクリックして、**ファイル1つだけ**を VS Code で開いてしまう失敗です。この開き方だと、サイドバーにフォルダの中身が出ず、VS Code の中のターミナルもプロジェクトとは関係ない場所（ホームフォルダなど）から始まります。そのまま `node index.mjs` や `npm run start` を打つと「見つからない」エラーになります。

NG（ファイルだけを開いた VS Code のターミナルで、そのまま実行）：

```
# ターミナルの現在地を確認すると、プロジェクトの外（ホームフォルダ）にいる
pwd
# → /c/Users/<ユーザー名>
node index.mjs
# → Error: Cannot find module 'C:\Users\<ユーザー名>\index.mjs'
```

OK（プロジェクトのフォルダごと開く）：

```
# プロジェクトのフォルダに移動してから、フォルダごと VS Code で開く
cd ~/Desktop/npm-practice
code .
```

GUI で開く場合は、VS Code のメニューの「ファイル」→「フォルダーを開く...」から、**フォルダ**を選びます。「プロジェクトはフォルダ単位で開く」と覚えておきましょう。

推奨する VS Code 設定

VS Code で `Ctrl + Shift + P`（Mac: `Cmd + Shift + P`）を押して「Preferences: Open Settings (JSON)」を選択し、以下の設定を追加します。

```
{
  // ファイル保存時に自動でフォーマットを実行
  "editor.formatOnSave": true,
  // デフォルトフォーマッターを Prettier に指定
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  // Tab1回あたりのスペース数を2に
  "editor.tabSize": 2,
  // 長い行を画面幅で折り返して表示
  "editor.wordWrap": "on",
  // 右側のミニマップ（縮小プレビュー）を非表示
  "editor.minimap.enabled": false,
  // 対応する括弧をペアごとに色分け
  "editor.bracketPairColorization.enabled": true,
  // カーソル位置の括弧ガイドラインを表示
  "editor.guides.bracketPairs": "active",
  // 別のファイルに切り替えた瞬間に自動保存
  "files.autoSave": "onFocusChange",
  // Windowsの統合ターミナルをGit Bashに
  "terminal.integrated.defaultProfile.windows": "Git Bash",
  // Emmet（HTML省略記法）を有効化する言語の追加設定
  "emmet.includeLanguages": {
    //   .js ファイルでも JSX 用の Emmet を使えるようにする
    "javascript": "javascriptreact"
  }
}
```

設定	効果
`formatOnSave`	ファイル保存時に自動フォーマット
`defaultFormatter`	Prettier をデフォルトフォーマッターに設定
`tabSize: 2`	インデント幅を2スペースに設定
`wordWrap`	長い行を折り返して表示
`autoSave`	フォーカスが外れたら自動保存
`defaultProfile.windows`	Windows のデフォルトターミナルを Git Bash に変更

---

4. Git の基礎

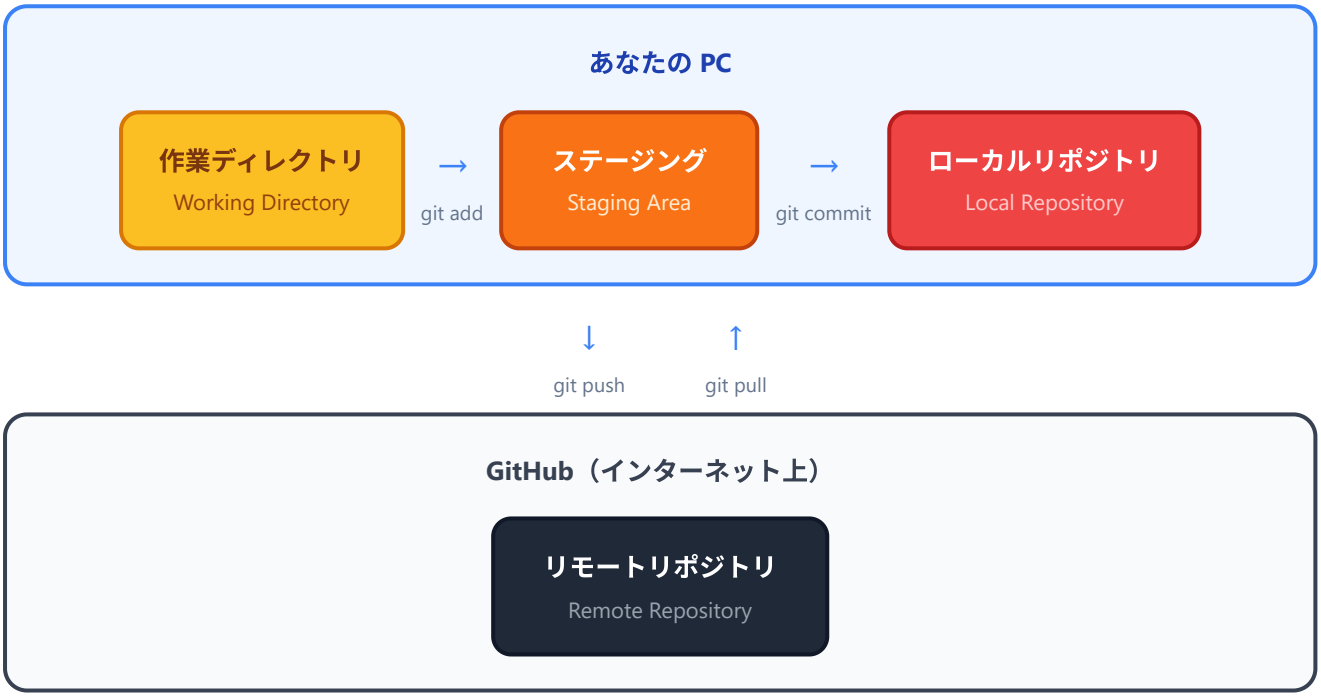
4.1 Git とは

Git は **バージョン管理システム**（Version Control System：VCS）です。ファイルの変更履歴を記録し、過去の状態に戻したり、複数人で同時に開発したりすることを可能にします。

> **Git用語の予習:**
>
> - **リポジトリ**（repository：略してリポ／レポ）：プロジェクトの履歴を全部入れておく「箱」。1プロジェクトに1つ作るのが基本。
> - **コミット**（commit）：「ここまでの変更を1つの履歴として保存する」操作・記録。スナップショット（瞬間記録）と考えるとイメージしやすい。
> - **ブランチ**（branch：枝）：履歴を枝分かれさせる仕組み。新機能を試すために本流から分岐 → うまくいったら本流に合流（マージ）する。
> - **リモート**（remote）：自分のPCの外（GitHubなど）にあるリポジトリ。それに対して自分のPC内のものは「ローカル」と呼ぶ。

なぜ Git が必要なのか

状況	Git なし	Git あり
コードを壊してしまった	元に戻せない	`git checkout` で復元可能
誰がいつ変更したか知りたい	わからない	`git log` で確認可能
新機能を試したい	メインのコードを壊すリスク	ブランチで安全に実験可能
複数人で開発	ファイルの上書き事故	マージ機能で安全に統合
コードを公開・共有したい	USBやメールで送る	GitHub にプッシュ



4.2 Git のインストール

Windows

1. Git for Windows にアクセス
2. インストーラーをダウンロードして実行
3. インストールオプション（以下のデフォルト推奨設定を確認）: - **Default editor**:「Use Visual Studio Code as Git's default editor」を選択（コミットメッセージなどを書くときに開かれるエディタの指定） - **Adjusting your PATH**:「Git from the command line and also from 3rd-party software」を選択（コマンドプロンプトからもGitを使えるようにする） - **Line ending conversions**:「Checkout Windows-style, commit Unix-style line endings」を選択（改行コードの自動変換設定。詳細は後述） - その他はデフォルトのまま「Next」を押してインストール

> 💡 **ヒント:** Git for Windows をインストールすると **Git Bash** というターミナルも一緒にインストールされます。Linux/Mac と同じコマンドが使えるので非常に便利です。

Mac

```
# 方法1: Xcode Command Line Tools（Apple純正の開発ツール一式）を入れる
# これに Git が同梱されている。初回はインストールダイアログが出る
xcode-select --install

# 方法2: Homebrew（Mac用パッケージマネージャー）で入れる（推奨: 最新版が使える）
# brew install <パッケージ名> でインストール
brew install git
```

Linux（Ubuntu/Debian）

```
# パッケージ一覧を最新化
sudo apt update
# git をインストール。-y で確認プロンプトを自動承認
sudo apt install git -y
```

バージョン確認

```
# Git のバージョンを表示
# --version はロングフラグ。多くのコマンドで共通
git --version
# 出力例: git version 2.43.0
```

4.3 Git の初期設定

インストール後、ユーザー名とメールアドレスを設定します。これはコミット（変更の記録）に「誰が変更したか」として記録される情報です。

```
# git config: Git の設定を読み書きするコマンド
# --global: PC全体（全プロジェクト共通）に保存。
#           付けないと現在のリポジトリだけの設定になる
# user.name: 設定キー（名前空間: 設定名）。コミット作者の名前を表す
# "..." : 設定値。スペースを含むので引用符必須
git config --global user.name "あなたの名前"

# メールアドレスを設定（GitHub アカウントと同じものを推奨）
# user.email: コミット作者のメールアドレス
git config --global user.email "your-email@example.com"

# 新規リポジトリ作成時のデフォルトブランチ名を main に設定
# 古い慣習では master だったが、現在は main が標準
git config --global init.defaultBranch main

# 現在の全グローバル設定を確認
# --list : 設定一覧を表示するフラグ
git config --global --list
```

▼ **実行結果**（`git config --global --list`。名前とメールアドレスは例）

```
user.name=Yamada Taro
user.email=taro@example.com
init.defaultbranch=main
```

3行とも表示されれば設定完了です（他の行が混ざっていても問題ありません）。設定のキー名は大文字・小文字を区別しないので、`init.defaultBranch` が `init.defaultbranch` と小文字で表示されても同じ意味です。

> **つまずきポイント:** この初期設定を飛ばしていきなり `git commit` すると、Git に「あなたは誰ですか？」と聞き返されて、コミットが**失敗**します。英語のメッセージに驚いて手が止まりがちですが、書いてあるとおりに名前とメールアドレスを設定すれば解決します。

NG（user.name と user.email を設定しないまま `git commit` を実行）：

```
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got '<ユーザー名>@<PC名>.(none)')
```

OK（設定してから、もう一度コミットする）：

```
# 名前とメールアドレスを設定する（自分のものに置き換える）
git config --global user.name "Yamada Taro"
git config --global user.email "taro@example.com"
# もう一度コミットする
git commit -m "最初のコミット"
```

メッセージの意味は「作者が不明です。あなたが誰か教えてください。`git config --global ...` を実行して設定してください」です。最後の `fatal:`（致命的エラー）の行は「メールアドレスを自動で推測できなかった」と言っています。

> **注意:** `user.email` は GitHub で公開されるコミット履歴に載ります。メールアドレスを公開したくない場合は、GitHub の「Settings」→「Emails」にある `（数字）+（ユーザー名）@users.noreply.github.com` 形式のアドレスを使うこともできます。

4.4 基本コマンド

リポジトリの作成と基本操作

Git の基本フローを「ファイル作成 → ステージ → コミット」の流れで体験します。各コマンドが何をしているか1行ずつ解説します。

> **「同じディレクトリにいる」って何?:** ターミナルは「いま開いているフォルダ」が常に1つあって、それを **カレントディレクトリ** と呼びます。`pwd` で確認できます。以下のコマンドは「`my-project` フォルダの中にいる状態」で実行することを前提にしています。フォルダから出てしまうと、別のフォルダに対して操作してしまうので注意。

```
# ------------------------------------------------------------------------
# (1) 新しい作業フォルダを作って、その中に移動する
# ------------------------------------------------------------------------
# mkdir = "make directory" の略。フォルダを新規作成するコマンド。
# 引数 my-project は作成するフォルダ名
mkdir my-project

# cd = "change directory" の略。カレントディレクトリ（今いる場所）を移動。
# これ以降のコマンドはこの my-project フォルダ内で動く
cd my-project


# ------------------------------------------------------------------------
# (2) Git リポジトリを初期化する
# ------------------------------------------------------------------------
# git init は「このフォルダをGit管理下にする」コマンド。
# 隠しフォルダ .git/ が作られ、ここに変更履歴が全部保存される。
# 一度実行すれば良い。再実行しても害はないが普通は不要
git init
# ▼ 期待する出力
# Initialized empty Git repository in /path/to/my-project/.git/


# ------------------------------------------------------------------------
# (3) サンプルファイルを1個作る
# ------------------------------------------------------------------------
# echo "..." は文字列をそのまま出力するコマンド。
# > README.md は「出力先をファイルに切り替える」リダイレクト演算子。
# 結果として「README.md という名前のファイルに "# My Project" と書く」操作になる。
# > は上書き、>> は追記。ここでは新規作成なので > を使用
echo "# My Project" > README.md


# ------------------------------------------------------------------------
# (4) 現在のリポジトリの状態を確認する
# ------------------------------------------------------------------------
# git status は「いま何が変わってる？」を教えてくれる。最頻出コマンド。
# 引数なしで何度実行してもファイルを変えない（安全に確認用に使える）
git status
# ▼ 期待する出力（要約）
# On branch main
# No commits yet
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#         README.md
# nothing added to commit but untracked files present (use "git add" to track)
#
# ▼ 「Untracked」= まだGitに認識されていない新規ファイル、の意味。
```

```
# ------------------------------------------------------------------------
# (5) ファイルを「ステージングエリア」に追加
# ------------------------------------------------------------------------
# Gitは2段階で変更を記録する。
#   ①ステージング (git add)  ← どの変更を記録するかを選ぶ
#   ②コミット    (git commit) ← 選んだ変更をスナップショットとして記録
# こうすることで「複数の変更のうち一部だけまとめてコミット」ができる。
# 引数 = 特定ファイル名。そのファイルだけ追加
git add README.md
# 参考: 引数を . にすると「カレントフォルダ以下の全変更」をまとめて追加できる。よく使う
# git add .


# ------------------------------------------------------------------------
# (6) コミットする = スナップショットを履歴に記録
# ------------------------------------------------------------------------
# -m "..." はコミットメッセージを指定するフラグ。message の m。
# なぜこの変更をしたかを短文で残す。必ず引用符で囲む。日本語OK。
# -m を付けないとデフォルトエディタ（vim/VS Code）が開いてメッセージ入力を求められる
git commit -m "最初のコミット: READMEを追加"
# ▼ 期待する出力（要約）
# [main (root-commit) 1a2b3c4] 最初のコミット: READMEを追加
#  1 file changed, 1 insertion(+)
#  create mode 100644 README.md


# ------------------------------------------------------------------------
# (7) コミット履歴を見る
# ------------------------------------------------------------------------
# git log だけだと1コミットあたり数行使うので画面が長くなる。
# --oneline で1行に圧縮表示できる。
# 通常表示: 1コミットあたり 作者/日時/メッセージ で4-5行
git log
# 1行表示: ハッシュとメッセージのみ。一覧性◎
git log --oneline
# ▼ 出力例（git log --oneline）
# 1a2b3c4 (HEAD -> main) 最初のコミット: READMEを追加
```

ファイルの「4つの状態」を図で理解する

Git の中で、ファイルは次の4つの状態のどれかにいます。写真撮影にたとえると、`git add` は「写したい物を**撮影台に並べる**」、`git commit` は「**シャッターを押して**、その瞬間をアルバムに貼る」操作です。撮影台に並べなかった物は、シャッターを押しても写りません。

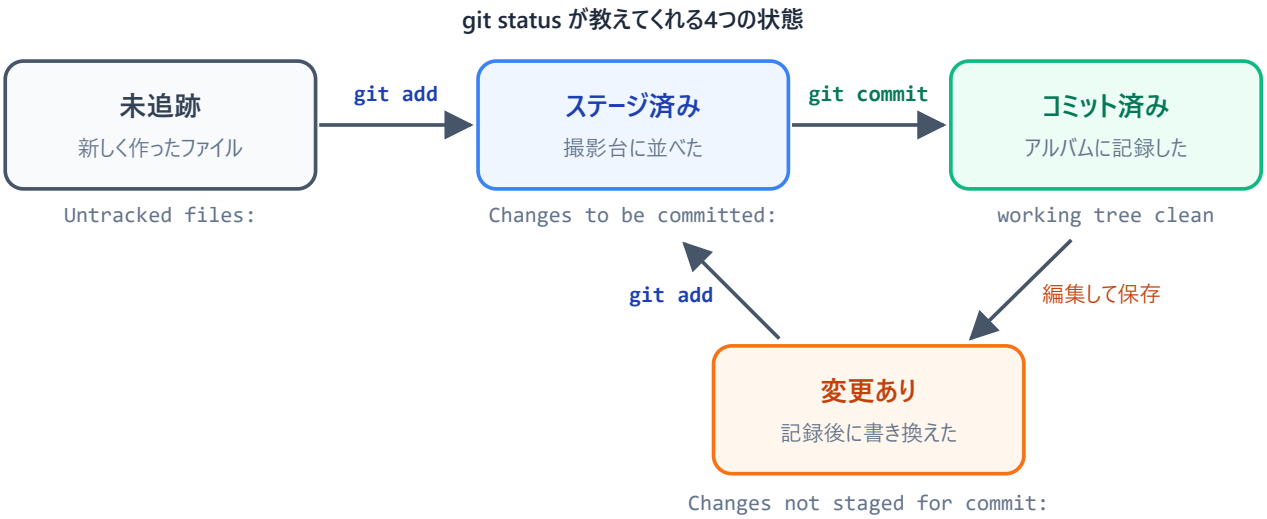


図1-9: ファイルの4つの状態と、状態を進めるコマンド。等幅文字は git status に表示される見出し

上の図の等幅文字（`Untracked files:` など）は、`git status` を実行したときに実際に表示される見出しです。**`git status` の見出しを見れば、ファイルが図のどこにいるか分かる**ので、迷ったらまず `git status` を打ちましょう。

やってみよう: 変更 → 差分を見る → 記録する（2回目のコミット）

上の (1)〜(7) で最初のコミットを作った `my-project` フォルダで、続けて「ファイルを書き換えて、もう一度記録する」流れを体験します。図1-9の「コミット済み → 変更あり → ステージ済み → コミット済み」の一周です。

> **▼ このコードがやること（先に日本語で）**: README.md に1行書き足し、Git に「どこが変わったか」を見せてもらってから、その変更を2回目のコミットとして記録します。最後に履歴が2件になったことを確かめます。

```
# README.md の最後に1行書き足す（>> は「末尾に追記」）
echo "- 読みたい本: リーダブルコード" >> README.md

# 状態を確認する（図1-9の「変更あり」になっているはず）
git status

# 何がどう変わったのか、中身の差分を見る
git diff

# 変更をステージに追加して、もう一度状態を確認する
git add README.md
git status

# ステージの内容をコミットとして記録する
git commit -m "読みたい本を1冊追加"

# 履歴を1行ずつ表示する
git log --oneline
```

▼ **実行結果**（Windows の Git Bash で実行。見やすいように各コマンドの前に `$ コマンド` の行を入れています）

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
$ git diff
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time
Git touches it
diff --git a/README.md b/README.md
index a2beefd..8280471 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
 # My Project
+- 読みたい本: リーダブルコード
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time
Git touches it
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   README.md

$ git commit -m "読みたい本を1冊追加"
[main d5f6dc9] 読みたい本を1冊追加
 1 file changed, 1 insertion(+)
$ git log --oneline
d5f6dc9 (HEAD -> main) 読みたい本を1冊追加
93e051b 最初のコミット: READMEを追加
```

`d5f6dc9` のような英数字（コミットの ID。ハッシュと呼びます）は、実行するたびに違う値になります。あなたの画面と一致しなくて大丈夫です。

▼ コードを1つずつ分解して解説

解説1: `git status` の「変更あり」表示

```
Changes not staged for commit:
    modified:   README.md
```

- 「コミット用にステージされていない変更があります。README.md が変更（modified）されています」という意味です。図1-9の「変更あり」の状態です。
- かっこ内の `use "git restore <file>..."` は「変更を取り消したいなら `git restore ファイル名` を使ってね」というヒントです。Git のメッセージには、次にやるべきことがよく書いてあります。

解説2: `git diff` の読み方

```
@@ -1 +1,2 @@
 # My Project
+- 読みたい本: リーダブルコード
```

- 行頭が `+` の行は**追加された行**、`-` の行は**削除された行**、空白の行は変わっていない行です。今回の `+- 読みたい本: …` は、先頭の `+` が「追加」の印で、その次の `-` は README.md に書いた箇条書きの記号そのものです。
- `@@ -1 +1,2 @@` は「変更前の1行目まわりが、変更後は1行目から2行ぶんになった」という場所の情報です。最初は読み飛ばしてかまいません。
- `warning: ... LF will be replaced by CRLF ...` は Windows だけで出る改行コードの注意です（4.6で説明します）。エラーではないので、作業はそのまま続けて大丈夫です。

解説3: ステージ後の `git status` と、コミット

```
Changes to be committed:
    modified:   README.md
```

- 「コミットされる予定の変更」＝撮影台に並んだ状態です。この状態で `git commit` すると、ここに並んだものだけが記録されます。
- コミット後の `1 file changed, 1 insertion(+)` は「1ファイルが変わり、1行が追加された」という記録の内容です。

> **つまずきポイント:** ファイルを編集したあと、`git add` を忘れて `git commit` だけ実行してしまう失敗です。撮影台に何も並べずにシャッターを押したのと同じで、**何も記録されません。**エラーっぽい赤字は出ないので、「コミットしたつもり」になりやすいのが怖いところです。

NG（編集後、`git add` せずに `git commit` を実行）:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

OK（先に `git add` してから `git commit`）：

```
# 変更をステージに並べてから
git add README.md
# コミットする
git commit -m "もう1冊追加"
```

```
[main 2633ef2] もう1冊追加
 1 file changed, 1 insertion(+)
```

NG の最後の行 `no changes added to commit`（コミットに追加された変更はありません）が「何も記録しなかったよ」の合図です。`[main 2633ef2] ...` のように**角かっこで始まる行が出たら成功**、と覚えておきましょう。

リモートリポジトリとの連携

```
# ------------------------------------------------------------------------
# (1) リモートリポジトリ「origin」を登録
# ------------------------------------------------------------------------
# git remote add <名前> <URL>: 別の場所にあるGitリポジトリに別名（origin）を付ける。
# origin は慣習的に「メインのリモート」を指す名前。GitHubで作ったリポジトリのURLを使う。
# 一度登録すれば次回以降は名前 origin だけで参照できる
git remote add origin https://github.com/ユーザー名/リポジトリ名.git


# ------------------------------------------------------------------------
# (2) ローカルのコミットをGitHubに送る（プッシュ）
# ------------------------------------------------------------------------
# git push <リモート名> <ブランチ名>
# -u は --set-upstream の短縮形。「次回からは git push だけで origin main に送る」設定。
# 初回プッシュ時にのみ付ければOK
git push -u origin main
# ▼ 出力例
# Enumerating objects: 3, done.
# Counting objects: 100% (3/3), done.
# Writing objects: 100% (3/3), 234 bytes | 234.00 KiB/s, done.
# Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
# To https://github.com/yourname/my-project.git
#  * [new branch]      main -> main
# branch 'main' set up to track 'origin/main'.


# ------------------------------------------------------------------------
# (3) GitHubの最新コミットをローカルに取り込む（プル）
# ------------------------------------------------------------------------
# 他の開発者やGitHub上で行われた変更を、自分のローカルに反映する。
# git pull = git fetch（取得）+ git merge（統合）の2段階を一気にやるコマンド
git pull origin main
```

よく使うコマンドの一覧

コマンド	説明
`git init`	リポジトリを初期化
`git status`	変更状態を確認
`git add <ファイル>`	ステージングエリアに追加
`git add .`	すべての変更をステージに追加
`git commit -m "メッセージ"`	変更をコミット
`git log --oneline`	コミット履歴を簡潔に表示
`git diff`	変更内容の差分を表示
`git branch`	ブランチ一覧を表示
`git checkout -b <名前>`	新しいブランチを作成して切り替え
`git push`	リモートにプッシュ
`git pull`	リモートからプル
`git clone <URL>`	リポジトリをコピー

4.5 GitHub アカウントの作成

1. GitHub にアクセス
2. 「Sign up」をクリック
3. メールアドレス、パスワード、ユーザー名を入力
4. メール認証を完了
5. 無料プラン（Free）を選択

SSH キーの設定（推奨）

パスワード入力なしで GitHub にプッシュできるように SSH キー（暗号化された電子鍵）を設定します。

> **SSHキーとは:** 「公開鍵」と「秘密鍵」のペア。公開鍵をGitHubに登録し、秘密鍵を自分のPCに置いておくと、毎回パスワードを打たずに安全に通信できる仕組み。

```
# SSH キーペア（公開鍵と秘密鍵）を生成する
# ssh-keygen: SSH鍵を作るコマンド
#   -t ed25519 : 鍵の種類（type）。ed25519 は最新の暗号方式で推奨
#   -C "..."  : コメント（Comment）。鍵の識別ラベル。メアドにするのが慣習
ssh-keygen -t ed25519 -C "your-email@example.com"
# 実行中に3回プロンプトが出る:
#   1) 保存先 → デフォルト ~/.ssh/id_ed25519 のままEnter
#   2) パスフレーズ → 空のままEnter（または任意のパス）
#   3) パスフレーズ確認 → 同じくEnter

# 公開鍵（公開しても安全な方の鍵）の中身を画面に表示
# cat: ファイル内容を表示するコマンド
# ~/.ssh/id_ed25519.pub : .pub が公開鍵の慣習的な拡張子
cat ~/.ssh/id_ed25519.pub
# 表示された ssh-ed25519 で始まる1行をすべてコピー（メアド部分含む）
```

GitHub での設定:

1. GitHub にログイン
2. 右上のアイコン →「Settings」
3. 左メニュー「SSH and GPG keys」
4. 「New SSH key」をクリック
5. Title: 任意の名前（例: "My PC"）
6. Key: コピーした公開鍵を貼り付け
7. 「Add SSH key」をクリック

```
# GitHubに接続できるかテスト
# ssh -T <ユーザー@ホスト> : テスト接続。-T はリモートシェル割り当て無効
# git@github.com : GitHubのSSH接続先（ユーザー名は固定で git）
ssh -T git@github.com
# 出力: Hi ユーザー名! You've successfully authenticated...
# 初回は「本当に接続しますか？」と聞かれるので yes と入力
```

4.6 Git トラブルシューティング

push 時に認証エラー

```
remote: Support for password authentication was removed
```

原因: GitHub はパスワード認証を廃止しました（2021年8月以降）。

解決方法: SSH キーを設定するか、Personal Access Token（PAT：個人用アクセストークン）を使用します。上記の SSH キーの設定を参照してください。

改行コードの警告（Windows）

```
warning: LF will be replaced by CRLF
```

> **改行コードの違い:** WindowsはCR+LF（ `\r\n` の2文字）、Mac/LinuxはLFのみ（ `\n` の1文字）で改行を表します。チームで開発するとここで衝突するため、Gitが自動変換します。

解決方法:

```
# Windows用設定: チェックアウト時に LF→CRLF へ自動変換し、コミット時にCRLF→LFへ戻す
# core.autocrlf : 改行コード自動変換の設定キー
#   true : Windowsスタイル変換（推奨）
#   input : Mac/Linux用設定
#   false : 変換しない
git config --global core.autocrlf true
```

コミットメッセージのエディタが vim で困る

```
# Gitがエディタを開く場面（git commit を -m なしで実行など）で使うエディタを変更
# core.editor : エディタを指定する設定キー
# "code --wait" : VS Code を起動し、--wait でファイルが閉じられるまで待つ
git config --global core.editor "code --wait"
```

---

5. ターミナル/コマンドラインの基礎

5.1 ターミナルとは

ターミナル（コマンドライン、シェル、CLI とも呼ばれます）は、テキストでコンピューターに指示を出すためのツールです。GUI（マウスでクリックする画面）では操作が複雑になるような作業も、コマンド一つで実行できます。

Web 開発ではターミナルを頻繁に使うため、基本的なコマンドを覚えておくことが重要です。

ターミナルの画面はこんな見た目

初めてターミナルを開くと、こんな画面が出てきます（黒や白の背景に、緑や白の文字）。

```
yuya@MyPC MINGW64 ~/Desktop
$ _
```

各部分の意味は次のとおりです。

部分	例	意味
ユーザー名	`yuya`	ログイン中のユーザーの名前
PC名	`MyPC`	あなたのコンピュータの名前
シェル種別	`MINGW64`	Git Bash の場合に表示される識別子。PowerShellなら別の表示
カレントディレクトリ	`~/Desktop`	「いま自分がいる場所」。`~` はホームフォルダ
プロンプト	`$` または `>`	「ここからコマンドを打ってね」という合図
カーソル	`_`（点滅）	入力位置を示す光る四角

> 「**カレントディレクトリ**」って何？: いまターミナルが「自分の現在地」として認識しているフォルダのことです。`pwd` コマンドで確認でき、`cd` コマンドで移動できます。Windowsのエクスプローラーで「現在開いているフォルダ」と同じ概念です。

コマンドを「実行する」とは？

たとえば `pwd` というコマンドを実行する手順は以下のとおりです。

1. プロンプト `$` の右側にカーソルが点滅していることを確認
2. キーボードで `pwd` と打つ
3. **Enterキー** を押す
4. すぐ下の行に**結果（出力）** が表示される
5. その下に新しいプロンプトが現れて、次のコマンドを待つ

▼ 実行例:

```
$ pwd
/c/Users/yuya/Desktop
$ _
```

ここで `pwd` の右下に表示された `/c/Users/yuya/Desktop` が、コマンドの**実行結果**です。これは「いまDesktopフォルダにいるよ」という意味です。

行頭の `$` や `>` はコピーしない

このチュートリアル中で出てくるサンプルコマンドの中には、行頭に `$` が付いているものがあります。

```
$ node -v
```

この `$` は「**これはターミナルに打つコマンドですよ**」という印で、**実際には打ちません**。打つのは `$` より右の `node -v` だけです。同じく PowerShell のサンプルで `PS C:\>` のような表示があったら、それも飾りです。

> **代表的なプロンプト記号:**
>
> - `$` : 一般ユーザー（Bash, Git Bash, zsh など）
> - `#` : 管理者（root）ユーザー
> - `>` : Windowsコマンドプロンプト
> - `PS C:\>` : PowerShell（ `PS` の後にカレントパスが続く）
>
> 本書では混乱を避けるため、コマンドだけを書いてプロンプト記号は省略するスタイルを基本にしています。

同じ `node -v` を3種類のターミナルで打つと、画面は次のようになります。行頭の表示（プロンプト）はバラバラですが、**自分で打つ部分（黄色）と結果は同じ**です。

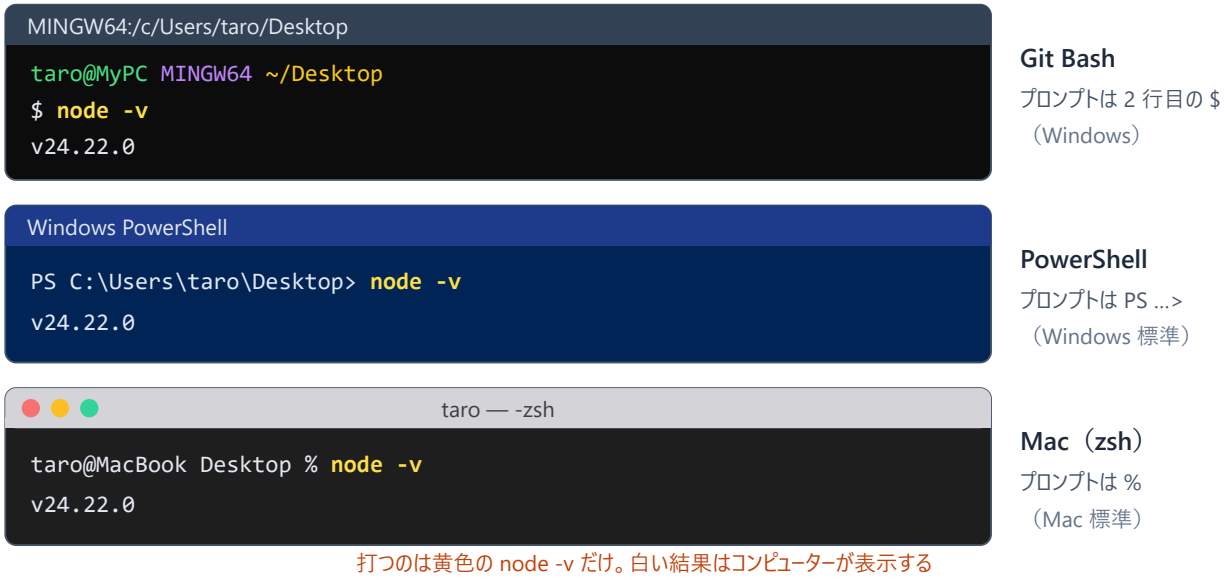


図1-10: ターミナルごとのプロンプトの違い（画面イメージ）。ユーザー名 taro・PC 名 MyPC は例

> **つまずきポイント:** 本や Web 記事のコマンドを、行頭の `$` ごとコピーして貼り付けてしまう失敗です。ターミナルは `$` を「`$` という名前のコマンド」だと思って探しに行き、見つからずにエラーになります。

NG（`$ node -v` を `$` ごと貼り付けた。1つ目の `$` はプロンプト、2つ目が貼り付けた `$`）:

```
$ $ node -v
bash: $: command not found
```

OK（`$` の右側だけを貼り付ける）：

```
$ node -v
v24.22.0
```

`bash: $: command not found` の `$:` の部分に注目してください。「`$` というコマンドは見つからない」と、**何が見つからなかったのか**をちゃんと教えてくれています。

5.2 Windows のターミナル選択肢

Windows には複数のターミナルがあります。

ターミナル	特徴	推奨度
Git Bash	Linux/Mac と同じコマンドが使える。Git に同梱	★★★★★
Windows Terminal	Microsoft 公式。複数のシェルをタブで使い分け可能	★★★★☆
PowerShell	Windows 標準。独自のコマンド体系	★★★☆☆
コマンドプロンプト (cmd)	旧来の Windows ターミナル。機能が限定的	★★☆☆☆

> 💡 **ヒント:** 本チュートリアルでは **Git Bash** の使用を推奨します。オンラインの記事やドキュメントの多くは Linux/Mac のコマンドで書かれており、Git Bash ならそれらをそのまま実行できます。

Git Bash の起動方法

- スタートメニューで「Git Bash」と検索して起動
- 任意のフォルダで右クリック →「Git Bash Here」
- VS Code のターミナルを Git Bash に設定（前述の設定で対応済み）

PowerShell と Git Bash のコマンド比較

操作	Git Bash / Mac / Linux	PowerShell
現在のフォルダ表示	`pwd`	`pwd` または `Get-Location`
ファイル一覧	`ls`	`ls` または `Get-ChildItem`
フォルダ移動	`cd folder`	`cd folder`
フォルダ作成	`mkdir folder`	`mkdir folder`
ファイル削除	`rm file.txt`	`Remove-Item file.txt`
フォルダ削除	`rm -rf folder`	`Remove-Item -Recurse folder`
ファイル内容表示	`cat file.txt`	`Get-Content file.txt`
テキスト検索	`grep "text" file`	`Select-String "text" file`
環境変数表示	`echo $PATH`	`$env:PATH`
画面クリア	`clear`	`cls` または `Clear-Host`

5.3 基本コマンド一覧

以下は、開発で頻繁に使うコマンドです（Git Bash / Mac / Linux 共通）。

ファイル・フォルダ操作

```
# 現在のフォルダ（ディレクトリ）のフルパスを表示
# pwd = "print working directory" の略
pwd
# 出力例: /c/Users/yuya/Desktop/education

# カレントディレクトリ内のファイル・フォルダ一覧を表示
# ls = "list" の略
ls
# 出力例: Documents  Downloads  Desktop

# 隠しファイル（.で始まるファイル）も含めた詳細一覧
# ls にフラグを2つ付ける書き方
#   -l : long format（詳細表示）。権限・所有者・サイズ・日時など
#   -a : all（隠しファイルも含む）
#   -la と並べて書いてもOK
ls -la
# 出力例:
# drwxr-xr-x  5 yuya  staff  160  1  1 12:00 .
# drwxr-xr-x  3 yuya  staff   96  1  1 12:00 ..
# -rw-r--r--  1 yuya  staff   50  1  1 12:00 .gitignore
# -rw-r--r--  1 yuya  staff  200  1  1 12:00 package.json

# フォルダの移動
# サブフォルダ Documents に入る
cd Documents
# .. は「一つ上」の意味。親フォルダへ移動
cd ..
# ~ はホームフォルダ。引数なしの cd でも同じ
cd ~
# 絶対パスで移動（Git BashでのWindowsパス表記）
cd /c/Users/yuya

# フォルダの作成
# 単一フォルダ作成
mkdir my-project
# -p は parents の略。途中のフォルダがなくても順に作成
# 普通の mkdir は親が無いとエラーになるが、-p は中間フォルダも自動生成
# src/components/ui を一気に作成
mkdir -p src/components/ui

# ファイルの作成
# touch : ファイルの「最終更新日時」を現在に変更するコマンド
# ファイルが存在しなければ「空ファイル」として作成される副作用がある
touch index.html
# echo "..." > file : echoの出力をファイルに上書き保存
echo "Hello" > hello.txt
```

```
# ファイルのコピー
# cp = "copy" の略
# 第1引数を第2引数の名前でコピー
cp file.txt file-backup.txt
# -r は recursive（再帰的）の略。フォルダごとコピーする場合に必須
cp -r src/ src-backup/

# ファイルの移動・リネーム
# mv = "move" の略。リネームも実は「同じフォルダ内への移動」扱い
# ファイル名変更
mv old-name.txt new-name.txt
# ファイルをフォルダに移動
mv file.txt Documents/

# ファイルの削除
# rm = "remove" の略。ゴミ箱を経由せず即削除
rm file.txt
# -r : recursive（フォルダ内も再帰的に削除）
# -f : force（確認なしで強制実行、存在しないファイルでもエラーにしない）
# 合わせて -rf で「フォルダごと黙って削除」になる。便利だが危険
rm -rf node_modules/
```

> ⚠️ **注意:** `rm -rf` は確認なしに完全削除されます。特に `rm -rf /` や `rm -rf ~` のような広範囲の削除は絶対に実行しないでください。取り返しがつきません。

やってみよう: フォルダを作って、中に入って、戻ってくる

上のコマンドを、実際の画面で通しで試してみましょう。Next.js のプロジェクトでよく見る `src/components` のような「フォルダの中のフォルダ」を作り、その中を行き来します。

> **▼ このコードがやること（先に日本語で）**：デスクトップに `practice` フォルダと、その中の `src/components` フォルダを一度に作ります。中に入って空のファイルを作り、深いフォルダまで移動してから、`cd ../..` で2段上に戻ります。そのたびに `pwd` と `ls` で「今どこにいて、何があるか」を確かめます。

```
# 1. 今いる場所を確認する（デスクトップにいる前提）
pwd
# 2. practice/src/components を途中のフォルダごと一度に作る
mkdir -p practice/src/components
# 3. practice の中に入って、場所を確認する
cd practice
pwd
# 4. 空のファイル README.md を作り、中身を一覧表示する
touch README.md
ls
# 5. src の中を一覧表示する（移動せずに、中だけのぞく）
ls src
# 6. 2段下の components まで一気に移動して、場所を確認する
cd src/components
pwd
# 7. 2段上（practice）に戻って、場所を確認する
cd ../..
pwd
# 8. 隠しファイルも含めて一覧表示する／フォルダには / を付けて表示する
ls -a
ls -F
```

▼ **実行結果**（Git Bash で実行。各コマンドの前に `$ コマンド` の行を入れています）

```
$ pwd
/c/Users/<ユーザー名>/Desktop
$ mkdir -p practice/src/components
$ cd practice
$ pwd
/c/Users/<ユーザー名>/Desktop/practice
$ touch README.md
$ ls
README.md  src
$ ls src
components
$ cd src/components
$ pwd
/c/Users/<ユーザー名>/Desktop/practice/src/components
$ cd ../..
$ pwd
/c/Users/<ユーザー名>/Desktop/practice
$ ls -a
.  ..  README.md  src
$ ls -F
README.md  src/
```

▼ コードを1つずつ分解して解説

解説1: 成功したコマンドは「何も言わない」

```
$ mkdir -p practice/src/components
$ cd practice
```

- `mkdir` や `cd`、`touch` は、成功すると**何も表示しません。**ターミナルの世界では「便りがないのは良い便り」で、何も出なければ成功です。
- 逆に、何か英語が表示されたら、それはたいていエラーか警告です。必ず読みましょう。

解説2: `cd src/components` と `cd ../..`

```
$ cd src/components
$ cd ../..
```

- `/` で区切ると、何段でも一度に移動できます。`src/components` は「src の中の components」です。
- `..` は「1つ上」なので、`../..` は「1つ上の、さらに1つ上」＝2段上です。`components` → `src` → `practice` と戻りました。

解説3: `ls -a` と `ls -F` の表示

```
$ ls -a
.  ..  README.md  src
$ ls -F
README.md  src/
```

- `-a` を付けると、`.`（今いるフォルダ自身）と `..`（1つ上のフォルダ）まで表示されます。`.gitignore` のような `.` で始まる隠しファイルも、`-a` で初めて見えます。
- `-F` を付けると、フォルダの名前の後ろに `/` が付きます。ファイルとフォルダの見分けがつかないときに便利です。

ファイル内容の表示

```
# ファイル全体を画面に出力
# cat = "concatenate" の略（本来は複数ファイル連結用コマンド）
cat package.json

# 長いファイルをページ単位でスクロール表示
# less は「ページャー」と呼ばれる対話的閲覧プログラム
# 操作: スペース=次ページ / b=前ページ / / で検索 / q で終了
less README.md
# (q キーで終了)

# ファイル先頭から指定行数だけ表示
# head = 「頭」の意味
# -n 10 : 表示する行数を10に指定（n は number の n）
head -n 10 server.js

# ファイル末尾から指定行数だけ表示
# tail = 「尻尾」の意味。ログの最新行を見るのによく使う
tail -n 10 server.log
```

その他の便利なコマンド

```
# 画面をクリアして上のほうの出力履歴を見えないところに送る
clear

# 過去に打ったコマンドの履歴を一覧表示
# 上矢印キー ↑ でも1つずつ呼び出せる
history

# 指定コマンドの実行ファイルがどこにあるか調べる
# which = 「どこ」の意味
which node
# 出力例: /home/yuya/.nvm/versions/node/v20.11.0/bin/node

# ファイル・フォルダを検索
# find <検索場所> <条件>
#   .            : カレントフォルダから検索開始
#   -name "*.js" : ファイル名が *.js（任意の名前.js）にマッチ
find . -name "*.js"

# ファイル内のテキストを検索
# grep <パターン> <ファイル>
grep "import" src/App.tsx
# -r : recursive。フォルダを指定するとその中を再帰的に検索
grep -r "TODO" src/
```

5.4 パス（Path）の基本

> **パスとは:** ファイルやフォルダの場所を表す住所のような文字列。書き方が2種類あります。

```
# 絶対パス（absolute path）：ルート（/）から始まる完全な住所
# どこから打っても同じ場所を指す
/c/Users/yuya/Desktop/education/my-project/src/App.tsx

# 相対パス（relative path）：現在のフォルダを基準にした書き方
# どこにいるかで指す先が変わる
# ./ は「ここ」。カレントフォルダ内の src/App.tsx
./src/App.tsx
# ../ は「一つ上」。親フォルダの中の other-project
../other-project/
# ../../ で「二つ上」。連結で何段でも遡れる
../../

# 特殊なパス記号
# チルダ。あなたのホームフォルダ（例: /c/Users/yuya）
~
# ピリオド単独。現在のフォルダ
.
# ピリオド2つ。一つ上のフォルダ
..
```

相対パスは「**今いる場所からの道案内**」です。同じ `../other-project` でも、今いる場所が変われば指す先も変わります。現在地が `book-app` のときの例を図にします。



図1-11: 現在地が book-app のときの相対パス。左の木が実際のフォルダ、右がそこを指す書き方（taro はユーザー名の例）

ちなみに、`App.tsx` を絶対パスで書くと `/c/Users/taro/Desktop/book-app/src/App.tsx`（Git Bash の場合）です。絶対パスは長いけれど、どこにいても同じ場所を指します。相対パスは短いけれど、現在地しだいで指す先が変

わります。

> **つまずきポイント:** フォルダ名に**空白（スペース）**が入っていると、ターミナルはそこで「別の値の区切り」だと判断してしまいます。`Program Files` のような名前を指定するときは、全体を `"` で囲みます。

NG（空白を含むパスを、そのまま `cd` に渡す）:

```
$ cd /c/Program Files
bash: cd: too many arguments
```

OK（`"` で囲んで、1つの値だと伝える）:

```
$ cd "/c/Program Files"
$ pwd
/c/Program Files
```

`too many arguments` は「値（引数）が多すぎる」という意味です。`cd` は行き先を1つだけ受け取るのに、空白で区切られて `/c/Program` と `Files` の2つを渡されたので困っています。こうした面倒を避けるため、**自分で作るプロジェクトのフォルダ名には、空白や日本語を使わず、半角英数字とハイフンだけ**（`book-app` など）にするのがおすすめです。

> **つまずきポイント:** Git Bash に、エクスプローラーからコピーした Windows 形式のパス（`\` 区切り）をそのまま貼り付ける失敗です。Git Bash では `\` が「次の1文字を特別扱いしない」という意味の記号なので、`\` が消えてしまいます。

NG（Windows 形式のパスをそのまま `cd` に渡す）:

```
$ cd C:\Windows\System32
bash: cd: C:WindowsSystem32: No such file or directory
```

OK（`/c/...` 形式で書く。または全体を `"` で囲む）:

```
$ cd /c/Windows/System32
$ pwd
/c/Windows/System32
$ cd "C:\Windows\System32"
$ pwd
/c/Windows/System32
```

エラーの `C:WindowsSystem32` を見ると、`\` が全部消えているのが分かります。Git Bash では、`C:\` は `/c/` と書くのが基本です。

5.5 ターミナルのトラブルシューティング

コマンドが見つからない

```
bash: some-command: command not found
```

確認手順:

```
# コマンドの実体ファイルがどこにあるか確認
# 何も出なければ未インストール、または PATH 未登録
which some-command

# 環境変数 PATH の中身を表示
# echo : 文字列出力
# $PATH : 変数 PATH の値を参照（$ が変数参照の記号）
echo $PATH

# PATH にコマンドの場所が含まれていない場合
# → 該当ツールを再インストールするか、PATH を設定する
```

日本語が文字化けする（Windows）

```
# Git Bash の場合、以下を ~/.bashrc に追加
# export : 環境変数を「子プロセスにも引き継がれる形」で設定するキーワード
# LANG : ロケール（言語と文字コード）の設定キー
# ja_JP.UTF-8 : 日本語(日本)、UTF-8文字コードという意味
export LANG=ja_JP.UTF-8
```

Tab 補完を活用する

```
# ファイル名やフォルダ名を途中まで入力して Tab キーを押すと残りが自動入力される
# タイポ防止と入力短縮の両方に役立つ最重要テクニック
cd Doc<Tab>
# → cd Documents/ に自動補完される

# 候補が複数ある場合は Tab を2回押すと一覧が表示される
cd D<Tab><Tab>
# → Desktop/  Documents/  Downloads/
```

6. 動作確認

すべてのツールが正しくインストールされたか、以下のチェックリストで確認しましょう。

6.1 確認コマンドの実行

ターミナル（Git Bash 推奨）を開いて、以下のコマンドを順番に実行してください。

```
# echo : 引数の文字列を画面に出力するコマンド。シェルスクリプトの「表示」担当
# "..." 内は1行の文字列として扱われる
echo "=== 開発環境チェック ==="
# 引数なしの echo は空行を出力。見やすさのため挿入
echo ""

echo "1. Node.js:"
# Node.js のバージョン表示
node -v
echo ""

echo "2. npm:"
# npm のバージョン表示
npm -v
echo ""

echo "3. Git:"
# Git のバージョン表示
git --version
echo ""

echo "4. VS Code:"
# VS Code のバージョン表示（3行出力）
code --version
echo ""

echo "=== チェック完了 ==="
```

▼ 期待する実行結果（バージョンの数字は時期によって変わります。以下は 2026年10月に Windows の Git Bash で実行したもの）：

```
=== 開発環境チェック ===

1. Node.js:
v24.22.0

2. npm:
11.19.1

3. Git:
git version 2.53.0.windows.2

4. VS Code:
1.129.1
8a7abeba6e03ea3af87bfbce9a1b7e48fed567b8
arm64

=== チェック完了 ===
```

▼ 結果の読み方:

- `node -v` の `v` は version の v。続く数字が Node.js の **メジャー.マイナー.パッチ** バージョン
- `npm -v` は数字のみ。10.x なら本書の手順で問題なく動作する
- `git --version` の末尾 `.windows.1` は Windows用ビルドの印
- `code --version` は3行出力される（バージョン番号、コミットハッシュ、CPU種別）

▼ もし `command not found` などのエラーが出たら:

- `node: command not found` → Node.jsが未インストール。1.2 章に戻ってインストール
- `git: command not found` → Gitが未インストール。4.2 章に戻ってインストール
- `code: command not found` → VS Codeのインストールはできているが、`code` コマンドのパスが通っていない。Mac は VS Codeのコマンドパレット（`Cmd+Shift+P`）で「Shell Command: Install 'code' command in PATH」を実行。Windows は VS Code を「PATHへの追加」にチェックを入れて再インストールし、ターミナルを開き直す

発展: チェックを Node.js のプログラムにまかせる

上の確認は、Node.js のプログラムにまとめて自動でやらせることもできます。表示がすっきりするうえ、「Node.js の版が古すぎないか」の判定までしてくれます。

> **▼ このコードがやること（先に日本語で）:** `node -v`・`npm -v`・`git --version`・`code --version` を順番に実行し、動いたものには `[OK]` と版を、動かなかったものには `[NG]` を表示します。最後に Node.js の版が本書の推奨（22 以上）を満たしているかを判定し、使えるツールの数を数えます。

1.3 で作った `node-practice` フォルダに、`check-env.mjs` という名前で保存します。

```
// 別のコマンドを実行して、その出力を受け取る道具を読み込む
import { execSync } from "node:child_process";

// 調べたいツールの一覧（表示名と、バージョンを確認するコマンド）
const tools = [
  { label: "Node.js", command: "node -v" },
  { label: "npm", command: "npm -v" },
  { label: "Git", command: "git --version" },
  { label: "VS Code", command: "code --version" },
];

let okCount = 0;

for (const tool of tools) {
  try {
    // コマンドを実行し、出力の1行目だけを取り出す
    const output = execSync(tool.command, { encoding: "utf8", stdio: "pipe" });
    const firstLine = output.trim().split("\n")[0];
    console.log(`[OK] ${tool.label.padEnd(8)} ${firstLine}`);
    okCount++;
  } catch {
    // コマンドが見つからない・失敗したときはここに来る
    console.log(`[NG] ${tool.label.padEnd(8)} 見つかりません（インストールと PATH を確認）`);
  }
}

// Node.js のメジャーバージョン（v24.22.0 なら 24）を数字で取り出して確認する
const major = Number(process.versions.node.split(".")[0]);
if (major >= 22) {
  console.log(`[OK] Node.js ${major} 系は本書の推奨（22 以上）を満たしています`);
} else {
  console.log(`[注意] Node.js ${major} 系は古すぎます。LTS 版に更新してください`);
}

console.log(`\n${okCount} / ${tools.length} 個のツールが使えます`);
```

```
# チェック用プログラムを実行する
node check-env.mjs
```

▼ **実行結果**（すべてそろっている場合）

```
[OK] Node.js  v24.22.0
[OK] npm      11.19.1
[OK] Git      git version 2.53.0.windows.2
[OK] VS Code  1.129.1
[OK] Node.js 24 系は本書の推奨（22 以上）を満たしています

4 / 4 個のツールが使えます
```

▼ **実行結果**（Git と VS Code が PATH に無い状態で実行した場合）

```
[OK] Node.js  v24.22.0
[OK] npm      11.19.1
[NG] Git      見つかりません（インストールと PATH を確認）
[NG] VS Code  見つかりません（インストールと PATH を確認）
[OK] Node.js 24 系は本書の推奨（22 以上）を満たしています

2 / 4 個のツールが使えます
```

▼ コードを1つずつ分解して解説

解説1: 道具の一覧を「配列」と「オブジェクト」で表す

```
const tools = [
  { label: "Node.js", command: "node -v" },
  { label: "npm", command: "npm -v" },
];
```

- `[ ... ]` は**配列**（値を順番に並べた入れ物）、`{ label: ..., command: ... }` は**オブジェクト**（名前つきの値をまとめた入れ物）です。どちらも第2章で詳しく学びます。
- 調べたいツールを増やしたいときは、この一覧に1行足すだけで済みます。チェックの処理を何度も書かなくてよいのが、一覧にしておく利点です。

解説2: 他のコマンドを実行する `execSync`

```
const output = execSync(tool.command, { encoding: "utf8", stdio: "pipe" });
```

- `execSync` は、ターミナルにコマンドを打ったのと同じことをプログラムから行い、表示された結果を文字列として受け取ります。
- `stdio: "pipe"` は「コマンドの表示（エラー表示も含む）を画面に直接出さず、プログラムが受け取る」という指定です。これが無いと、エラーのときに英語のメッセージが画面に混ざります。

解説3: 失敗に備える `try` / `catch`

```
try {
  // うまくいけばここを実行
} catch {
  // 失敗したらここを実行
}
```

- コマンドが見つからないと `execSync` は**エラーを投げ**、そのままだとプログラム全体が止まってしまいます。
- `try { ... } catch { ... }` で囲むと、「失敗したら `catch` の中を実行して、次に進む」ことができます。だから1つが NG でも、残りのツールのチェックを続けられます。

解説4: 版の数字を取り出して比べる

```
const major = Number(process.versions.node.split(".")[0]);
```

- `process.versions.node` は `"24.22.0"` のような文字列（`v` は付きません）です。`split(".")` で `["24", "22", "0"]` に分け、`[0]` で先頭の `"24"` を取り出します。
- `Number()` で文字列の `"24"` を数の `24` に変換してから、`>= 22`（22 以上か）で比べます。文字列のままだと正しく大小を比べられないためです。

6.2 チェックリスト

以下のすべてにチェックが入れば、環境構築は完了です。

- [ ] **Node.js** がインストールされている（`node -v` で `v22.0.0` 以上が表示される。推奨は `v24`）
- [ ] **npm** がインストールされている（`npm -v` でバージョンが表示される）
- [ ] **Git** がインストールされている（`git --version` でバージョンが表示される）
- [ ] **Git の初期設定** が完了している（`git config --global user.name` で名前が表示される）
- [ ] **VS Code** がインストールされている（`code --version` でバージョンが表示される）
- [ ] **VS Code の日本語化** が完了している（メニューが日本語で表示される）
- [ ] **VS Code の必須拡張機能** がインストールされている（ESLint, Prettier, React snippets）
- [ ] **GitHub アカウント** を作成済み
- [ ] **ターミナル** で基本コマンドが実行できる（`pwd`, `ls`, `cd` など）

6.3 簡単なテスト

最後に、環境が正しく動作するかを実際に試してみましょう。

```
# 1. テスト用フォルダを作成して移動
# mkdir : フォルダ作成
# ~/Desktop : ホーム配下のDesktop。Mac/Linux/Git Bashで共通
mkdir ~/Desktop/env-test
cd ~/Desktop/env-test

# 2. Node.js プロジェクトを初期化
# npm init で package.json を作成
# -y : すべての質問にデフォルト値(yes)で答える
npm init -y

# 3. 簡単な JavaScript ファイルを作成
# cat > <ファイル> << 'EOF' ... EOF はヒアドキュメント構文
#   cat   : 引数なしだと標準入力をそのまま出力
#   > file: 出力をファイルに保存
#   <<    : ヒアドキュメント開始（複数行入力を流し込む）
#   'EOF' : 終端マーカー。シングルクォートで囲むと $変数を展開しない（ここでは ${nodeVersion} などをそのまま書きたいので必須）
cat > hello.js << 'EOF'
const message = "環境構築が完了しました！";
const nodeVersion = process.version;
const platform = process.platform;

console.log("====================================");
console.log(message);
console.log(`Node.js バージョン: ${nodeVersion}`);
console.log(`プラットフォーム: ${platform}`);
console.log("====================================");
console.log("");
console.log("次の章では、プロジェクトの作成を始めます。");
console.log("お疲れ様でした！");
EOF

# 4. 作成した JavaScript ファイルを Node.js で実行
# node <ファイル名> : 指定ファイルを実行
node hello.js
```

以下のような出力が表示されれば成功です。

```
====================================
環境構築が完了しました！
Node.js バージョン: v24.22.0
プラットフォーム: win32
====================================

次の章では、プロジェクトの作成を始めます。
お疲れ様でした！
```

```
# 5. Git の動作確認
# 念のためテストフォルダに移動
cd ~/Desktop/env-test
# このフォルダを Git 管理下にする（.git/ ができる）
git init
# カレントフォルダ以下の全変更をステージに追加
git add .
# 履歴にコミットを記録
git commit -m "環境テスト: 初回コミット"
# コミット履歴を1行表示で確認
git log --oneline

# 6. テストフォルダを削除（任意）
# rm -rf : 中身ごとフォルダを強制削除。実行前に必ずパスを確認
# 削除対象から外に出る（中にいると削除に失敗する場合あり）
cd ~/Desktop
rm -rf env-test
```

おまけ: TypeScript のファイルもそのまま動かしてみる

次の第2章では TypeScript（タイプスクリプト：JavaScript に「型」を足した言語）を学びます。Node.js 24 は、型の書かれた `.ts` ファイルを**そのまま**実行できます（Node.js 22 でも 22.18 以降なら同じです）。ひと足先に試してみましょう。

> **▼ このコードがやること（先に日本語で）**：「本」のデータの形（タイトルと読書ステータス）を型として決め、その形に合わせた本を3冊用意して、1冊ずつ「読了／読書中／読みたい」を表示します。読書ステータスの3つの値は、この本で作る書籍管理アプリと同じものです。

1.3 で作った `node-practice` フォルダに `hello.ts` という名前で保存し、ターミナルでもそのフォルダに移動してから実行します。

```ts
// 読書ステータスとして使ってよい値を、この3つだけに限定する
type Status = "want_to_read" | "reading" | "completed";

// 本のデータの「形」を決める（TypeScript の型。第2章で詳しく学びます）
type Book = {
  title: string;
  status: Status;
};

// ステータスの値と、画面に出す日本語の対応表
const labels = {
  want_to_read: "読みたい",
  reading: "読書中",
  completed: "読了",
};

// 決めた形に合わせて、本のデータを3冊ぶん作る
const books: Book[] = [
  { title: "リーダブルコード", status: "completed" },
  { title: "達人プログラマー", status: "reading" },
  { title: "プログラミング TypeScript", status: "want_to_read" },
];

// 1冊ずつ取り出して、タイトルとステータスを表示する
for (const book of books) {
  console.log(`${book.title}: ${labels[book.status]}`);
}
```

```sh
# .ts ファイルも node コマンドでそのまま実行できる
node hello.ts
```

▼ **実行結果**（Node.js v24.22.0）

```
リーダブルコード: 読了
達人プログラマー: 読書中
プログラミング TypeScript: 読みたい
```

▼ コードを1つずつ分解して解説

解説1: 使ってよい値を限定する `type Status`

```ts
type Status = "want_to_read" | "reading" | "completed";
```

- `|`（縦棒）は「または」の意味です。「`Status` とは、`"want_to_read"` か `"reading"` か `"completed"` のどれか」という約束を、`Status` という名前で決めています。
- こう決めておくと、`"finished"` のような約束にない値を書いたとき、VS Code が「その値は使えません」と赤線で教えてくれます。打ち間違いを書いている最中に見つけられるのが、型の一番うれしい点です。

解説2: `type Book` でデータの形を決める

```
type Book = {
  title: string;
  status: Status;
};
```

- 「本とは、文字列（`string`）の `title` と、`Status` のどれかの値を持つ `status` から成るもの」という**形の約束**を、`Book` という名前で決めています。
- `const books: Book[]` の `: Book[]` は「この変数には `Book` の形のデータが並んだ配列を入れます」という宣言です。

解説3: 対応表から日本語を引く `labels[book.status]`

```
console.log(`${book.title}: ${labels[book.status]}`);
```

- `labels` は「英語の値 → 日本語」の対応表（オブジェクト）です。`labels["completed"]` と書くと `"読了"` が取り出せます。
- `book.status` の中身は本ごとに違うので、`labels[book.status]` と書けば、その本のステータスに合った日本語が取り出せます。データには英語の値を保存し、画面に出すときだけ日本語に変える、という分け方は、第7章以降のアプリでも使います。

> **注意:** Node.js は `.ts` ファイルを実行するとき、型の部分を**取り除いて**から動かすだけで、「型の約束が守られているか」のチェックはしません。型のチェックは、第2章以降で使う TypeScript 本体（`tsc`）や VS Code、Next.js が行います。

6.4 問題が解決しない場合

上記の手順で問題が解決しない場合は、以下を試してください。

1. **PC を再起動する** — 環境変数の変更が反映されていない可能性があります
2. **ターミナルを新しく開き直す** — 設定の再読み込みが必要な場合があります
3. **ツールを再インストールする** — インストール時にオプションを間違えた可能性があります
4. **エラーメッセージで検索する** — エラーメッセージをそのまま Google で検索すると、解決策が見つかることが多いです

いちばん多い「command not found（コマンドが見つからない）」系のエラーは、次の順番で確かめると効率よく原因にたどり着けます。**手軽に確かめられるものから順に**並べてあります。

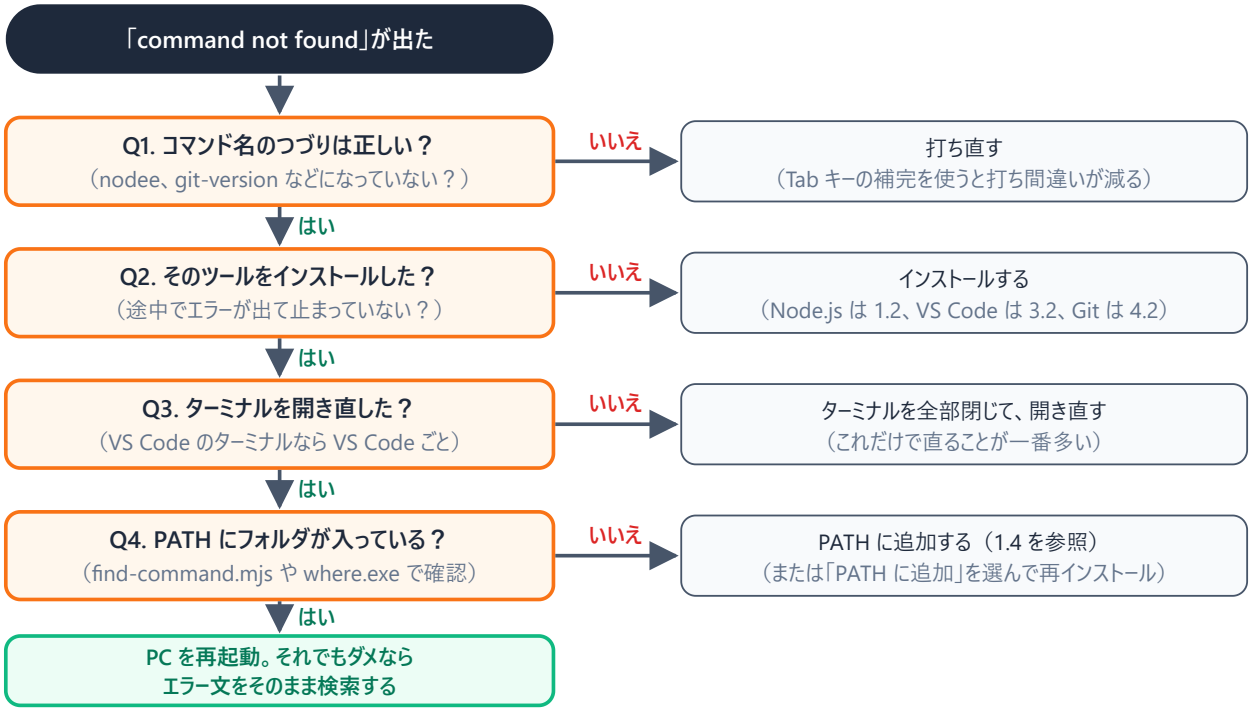


図1-12:「command not found」が出たときに確認する順番。上から順に、手軽なものから確かめる

まとめ

この章では、以下のツールをインストールし、設定しました。

ツール	用途	確認コマンド
Node.js	JavaScript の実行環境	`node -v`
npm	パッケージマネージャー	`npm -v`
VS Code	コードエディタ	`code --version`
Git	バージョン管理	`git --version`
GitHub	コードの共有・ホスティング	ブラウザでログイン確認

次の第2章では、アプリを書くための言語である TypeScript の基礎を学びます。そのあと React・Next.js・Supabase の基礎を順に学び、第6章で、この章で用意したツールを使って書籍管理アプリのプロジェクトを実際に作成します。

練習問題（解答・解説つき）

この章の内容を、手と頭を動かして確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んでください。コマンドやプログラムの問題は、実際にターミナルで試すのがいちばんの近道です。

問題1:「認識されません」と言われたら

友人が「Node.js をインストールしたのに、PowerShell で `node -v` と打つと『用語 'node' は、コマンドレット、関数、…として認識されません』と出る」と困っています。確認すべきことを、**手軽に確かめられる順に3つ**挙げてください。

問題2: `^` が受け入れる範囲

package.json に `"ms": "^2.1.3"` と書かれています。npm がインストールしてよいバージョンを、次からすべて選んでください。

- (a) `2.1.2`
- (b) `2.1.9`
- (c) `2.5.0`
- (d) `3.0.0`

問題3: Git に入れないもの

次のうち、Git に**入れない（コミットしない）**ものをすべて選び、それぞれ理由を答えてください。

- (a) `package.json`
- (b) `package-lock.json`
- (c) `node_modules/`
- (d) `index.mjs`
- (e) `.env.local`（第6章で作る、Supabase への接続情報を書くファイル）

問題4: `cd` のあとの現在地

Git Bash で、現在地が `/c/Users/<ユーザー名>/Desktop/book-app/src` です。

1. `cd ..` を実行したあとに `pwd` を実行すると、何と表示されますか。
2. 続けて `cd ../other-project` を実行したあとに `pwd` を実行すると、何と表示されますか（`other-project` フォルダは `Desktop` の中にあるものとします）。

問題5: Git のコマンドの順番

新しく作ったフォルダの中に `README.md` を作り、それを最初のコミットとして記録したいと思います。次のコマンドを、正しい順番に並べ替えてください（`git status` はいつ何回使ってもかまいません）。

- ア: `git commit -m "最初のコミット"`
- イ: `git init`
- ウ: `git add README.md`

問題6: 自分でプログラムを書く

`node-practice` フォルダに `whereami.mjs` というファイルを作り、次の2つを表示するプログラムを書いて、実行してください。

1. Node.js のバージョン
2. いまターミナルがいるフォルダ（カレントディレクトリ）

ヒント: 1.3 で使った `process.version` と、カレントディレクトリを返す `process.cwd()` を使います。

問題7: エラー文を読む

`npm run dev` を実行したら、次のエラーが表示されました。原因と、直すための手順を答えてください。

```
npm error code ENOENT
npm error syscall open
npm error path C:\Users\taro\package.json
npm error errno -4058
npm error enoent Could not read package.json: Error: ENOENT: no such file or directory,
open 'C:\Users\taro\package.json'
```

解答と解説

問題1の解答

1. **つづりの確認**: `node` を `nodee` や `mode` と打ち間違えていないか。
2. **ターミナルを開き直す**: インストール前から開いていたターミナルは、古い PATH のまま動いています。いったん全部閉じて、新しく開いてから `node -v` を試します（VS Code のターミナルなら VS Code ごと再起動）。
3. **PATH に Node.js のフォルダが入っているか**: `$env:PATH -split ";"` で一覧を表示する、または 1.4 の `find-command.mjs` で `node` を探します。入っていなければ、PATH に追加するか、Node.js（または nvm）を入れ直します。

解説: 図1-12の順番です。ターミナルは PATH に並んだフォルダを上から順に探し、どこにも `node.exe` が無いとこのエラーを出します（図1-5）。中でも「インストール後にターミナルを開き直していない」が圧倒的に多い原因です。PC の再起動は、これらを確かめたあとの最終手段です。

問題2の解答

(b) と (c) です。

解説: `^2.1.3` は「メジャー番号（先頭の 2）は変えず、`2.1.3` 以上」という範囲です（図1-7）。(a) `2.1.2` は下限の `2.1.3` より古いので対象外、(d) `3.0.0` はメジャー番号が変わるので対象外です。2.3 で使った `semver` で確かめられます。

```
# ^2.1.3 の範囲に入る版だけを表示する
npx --yes semver -r "^2.1.3" 2.1.2 2.1.9 2.5.0 3.0.0
```

▼ 実行結果

```
2.1.9
2.5.0
```

問題3の解答

(c) `node_modules/` と (e) `.env.local` です。

解説:

- (c) `node_modules/` は、`npm install` を実行すれば package.json と package-lock.json からいつでも元どおりに作り直せる「冷蔵庫の中身」です（図1-6）。ファイル数がとても多く重いので、Git には入れません。
- (e) `.env.local` には、外部サービスへの接続情報など、他人に見せるべきでない値を書きます。GitHub に上げると世界中に公開されてしまうおそれがあるので、Git には入れません。第6章で使う `create-next-app` が作る `.gitignore` は、最初からこの2つを除外しています。
- (a) と (b) は、全員が同じ部品を同じ版でそろえるために**必ず**コミットします。(d) は自分で書いたコードそのものなので、もちろんコミットします。

問題4の解答

1. `/c/Users/<ユーザー名>/Desktop/book-app`
2. `/c/Users/<ユーザー名>/Desktop/other-project`

解説: `..` は「1つ上」です。`src` から1つ上に出ると `book-app` です。そこからさらに `../other-project` で「1つ上（`Desktop`）に出てから、その中の `other-project` へ」移動します（図1-11）。実際に試した結果は次のとおりです。

▼ 実行結果（Git Bash。ユーザー名は伏せています）

```
$ pwd
/c/Users/<ユーザー名>/Desktop/book-app/src
$ cd ..
$ pwd
/c/Users/<ユーザー名>/Desktop/book-app
$ cd ../other-project
$ pwd
/c/Users/<ユーザー名>/Desktop/other-project
```

問題5の解答

イ → ウ → ア（`git init` → `git add README.md` → `git commit -m "最初のコミット"`）です。

解説:

- 最初に `git init` で、そのフォルダを Git の記録の対象にします。これをしないと、`git add` も `git commit` も「ここは Git のリポジトリではありません」というエラーになります。
- 次に `git add` で、記録したいファイルを撮影台（ステージ）に並べます。
- 最後に `git commit` でシャッターを押します。`git add` を飛ばすと、4.4 のつまずきポイントのとおり**何も記録されません**（図1-9）。
- 迷ったら、各手順の間で `git status` を打って、ファイルが図1-9のどの状態にいるかを確かめましょう。

問題6の解答

```
// Node.js のバージョンを表示する（process は Node.js が最初から用意している情報のかたまり）
console.log("Node.js のバージョン:", process.version);

// いまターミナルがいるフォルダ（カレントディレクトリ）を表示する
console.log("いまいる場所:", process.cwd());
```

```
# node-practice フォルダの中で実行する
node whereami.mjs
```

▼ **実行結果**（Windows で実行。ユーザー名は伏せています）

```
Node.js のバージョン: v24.22.0
いまいる場所: C:\Users\<ユーザー名>\Desktop\node-practice
```

解説: `process.cwd()` の `cwd` は current working directory（カレント・ワーキング・ディレクトリ＝現在の作業フォルダ）の略で、ターミナルの `pwd` と同じ情報を返します。Node.js が表示するのは Windows 形式のパス（`\` 区切り）なので、Git Bash の `pwd`（`/c/...` 形式）とは見た目が違いますが、指している場所は同じです。試しに `cd ..` で1つ上の `Desktop` に移ってから、`node node-practice/whereami.mjs` のようにファイルの場所を指定して実行してみてください。同じプログラムなのに、「いまいる場所」の表示が `Desktop` に変わります。プログラムの置き場所ではなく、**ターミナルの現在地**が表示されるからです。

問題7の解答

原因: `package.json` が無いフォルダ（ここではホームフォルダの `C:\Users\taro`）で `npm run dev` を実行しています。エラー文の `Could not read package.json`（package.json を読めなかった）と、`path` の行に書かれた場所がその証拠です。

直す手順:

```
# 1. 今いる場所を確認する
pwd
# 2. package.json がある、プロジェクトのフォルダに移動する（フォルダ名は自分のものに）
cd ~/Desktop/book-app
# 3. package.json があることを目で確かめる
ls
# 4. あらためて実行する
npm run dev
```

解説: `ENOENT` は「そのファイルやフォルダが無い」という意味の定番エラーです（2.5 のつまずきポイント）。npm のコマンドは、**今いるフォルダの package.json**を読んで動くので、まず `pwd` で現在地を確かめる習慣をつけましょう。VS Code でプロジェクトのフォルダを開き、VS Code の中のターミナルを使うと、最初からプロジェクトのフォルダにいる状態で始められるので、この失敗が起きにくくなります。

> 💡 **ヒント:** 環境構築は最初の一回だけです。一度セットアップが完了すれば、今後の章ではコードを書くことに集中できます。ここまでお疲れ様でした！

次の章へ: 第2章 TypeScript の基礎 →